

# **WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence**

**College / University Name: Arifa Institute of Technology**

**College Code: 8226**

**Team ID :SWTID-2026-9168**

## **TEAM DETAILS**

**Team Leader: Pranesh B**

**Registered Email ID:praneshkiruthvik23@gmail.com**

**Team Member 1:Kaviya M**

**Registered Email ID:kaviya.valarmathi2005@gmail.com**

**Team Member 2:Praveena D**

**Registered Email ID:praveenadharumarasujayan@gmail.com**

**Team Member 3:Mohamed Arshath M**

**Registered Email ID:arshath2k55@gmail.com**

## **User Story:**

WhatsNext Vision Motors is revolutionizing its customer experience and operational efficiency with a cutting-edge Salesforce CRM implementation. The project streamlines the vehicle ordering process by auto-assigning orders to the nearest dealer based on customer location and preventing orders for out-of-stock vehicles. Automated workflows update order statuses dynamically and send scheduled email reminders for test drives. Key technical implementations include Apex triggers for stock validation, batch jobs for stock updates, and scheduled Apex for automated order processing. This initiative enhances customer satisfaction, improves order accuracy, and boosts overall operational efficiency.

## **1. INTRODUCTION**

### **1.1 Project Overview :**

WhatNext Vision Motors implemented a Salesforce-based CRM solution to modernize vehicle ordering and dealership operations. The system automates vehicle order placement, validates stock availability, assigns orders to the nearest dealer, schedules test drive reminders, and updates order statuses dynamically. This transformation replaces manual and disconnected processes with a centralized, automated platform, improving operational efficiency and customer satisfaction.

## 1.2 Purpose :

The purpose of this project is to automate vehicle ordering, stock management, dealer assignment, and customer engagement using Salesforce CRM. The system aims to reduce manual intervention, eliminate order errors, provide real-time tracking, and enhance overall business productivity while delivering a seamless customer experience.

## 2. IDEATION PHASE

### 2.1 Problem Statement (Customer Problem Statement Template)

| PS   | I am<br>(Customer)    | I'm trying<br>to            | But                     | Because                         | Which<br>makes me<br>feel |
|------|-----------------------|-----------------------------|-------------------------|---------------------------------|---------------------------|
| PS-1 | Operations<br>Manager | manage<br>vehicle<br>orders | stock info is<br>manual | systems are<br>disconnecte<br>d | stressed                  |
| PS-2 | Customer              | place<br>vehicle<br>orders  | availability<br>unclear | delayed<br>updates              | uncertain                 |

## 2.2 Empathy Map Canvas

**User Persona:** Operations Manager

- **Think & Feel:** Concerned about wrong orders & stock errors
- **Hear:** Dealer and customer complaints
- **See:** Registers & spreadsheets
- **Say & Do:** Manual follow-ups
- **Pain:** Delays & cancellations
- **Gain:** Automation & dashboards

## **2.3 Brainstorming & Idea Prioritization**

### **Ideas Considered**

- Manual registers
- Spreadsheet tracking
- Standalone portal
- Salesforce CRM

### **Final Selection**

Salesforce CRM selected due to:

- Automation
- Scalability
- Real-time visibility

### **3. REQUIREMENT ANALYSIS PHASE**

#### **3.1 Customer Journey Map**

Stages:

- Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting

## 3.2 Solution Requirements

### Functional Requirements

| FR No | Requirement             |
|-------|-------------------------|
| FR-1  | Vehicle management      |
| FR-2  | Dealer management       |
| FR-3  | Customer order creation |
| FR-4  | Stock validation        |
| FR-5  | Dealer auto-assignment  |
| FR-6  | Test drive scheduling   |
| FR-7  | Reports & dashboards    |

### Non-Functional Requirements

| NFR         | Description                      |
|-------------|----------------------------------|
| Usability   | Lightning UI for easy navigation |
| Security    | Role-based access control        |
| Reliability | Accurate automation              |
| Performance | Fast order processing            |

|              |                      |
|--------------|----------------------|
| Availability | 24/7 cloud access    |
| Scalability  | Multi-dealer support |

### 3.3 Data Flow Diagram

Flow:

Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports

### 3.4 Technology Stack

| Layer      | Technology                 |
|------------|----------------------------|
| UI         | Salesforce Lightning       |
| Logic      | Salesforce Flows           |
| Database   | Salesforce Objects         |
| Automation | Validation Rules, Triggers |
| Processing | Batch Apex, Scheduled Apex |
| Reporting  | Reports & Dashboards       |

## 4. PROJECT DESIGN PHASE

### 4.1 Problem–Solution Fit

The system eliminates fragmented operations by integrating automated stock validation, dealer assignment, and order processing within Salesforce.

### 4.2 Proposed Solution

#### Functional Requirements

| Parameter      | Description                 |
|----------------|-----------------------------|
| Problem        | Manual vehicle ordering     |
| Solution       | Salesforce CRM              |
| Innovation     | Automated dealer assignment |
| Impact         | Faster service              |
| Business Model | Dealer network              |
| Scalability    | Multi-location support      |

### 4.3 Solution Architecture

Components:

- Vehicle Object
- Dealer Object
- Customer Object
- Order Object

- Test Drive Object
- Service Request Object
- Flows
- Apex Triggers
- Batch Jobs
- Reports
- Dashboards

## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

#### Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

| Sprint   | Functional Requirement (Epic) | User Story Number | User Story / Task                                                                                                                                                         | Story Points | Priority | Team Members      |
|----------|-------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|----------|-------------------|
| Sprint-1 | Developer Setup               | USN-1             | As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order management system.              | 3            | High     | Pranesh B         |
| Sprint-2 | Data Modeling                 | USN-2             | As an admin, I want to create Vehicle, Dealer, Customer, Order,                                                                                                           | 5            | High     | Kaviya M          |
| Sprint   | Functional Requirement (Epic) | User Story Number | User Story / Task                                                                                                                                                         | Story Points | Priority | Team Members      |
|          |                               |                   | and Service Request objects with proper relationships so that all business data is stored and managed systematically.                                                     |              |          |                   |
| Sprint-2 | Data Modeling                 | USN-3             | As a staff member, I want dedicated tabs and a Lightning App for all modules so that I can easily navigate and manage vehicle orders and dealer operations.               | 5            | High     | Praveena D        |
| Sprint-3 | Automation                    | USN-4             | As a system, I want to validate stock availability, mandatory fields, and order inputs so that invalid or incomplete vehicle orders are prevented.                        | 3            | High     | Mohamed Arshath M |
| Sprint-3 | Automation                    | USN-5             | As a sales coordinator, I want automated flows to assign the nearest dealer and update order status so that processing becomes faster and requires minimal manual effort. | 3            | High     | Mohamed Arshath M |

|          |            |       |                                                                                                                                                                                |   |        |                   |
|----------|------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|--------|-------------------|
| Sprint-4 | Apex       | USN-6 | As a system, I want Apex triggers to automatically perform actions such as order confirmation and dealer assignment so that critical processes occur instantly and accurately. | 5 | High   | Pranesh B         |
| Sprint-4 | Apex       | USN-7 | As an inventory manager, I want batch jobs to update vehicle stock levels automatically so that inventory data remains accurate and up to date.                                | 5 | High   | Kaviya M          |
| Sprint-5 | Reports    | USN-8 | As a manager, I want reports showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.                         | 4 | High   | Praveena D        |
| Sprint-6 | Dashboards | USN-9 | As a manager, I want dashboards showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.                      | 4 | Medium | Mohamed Arshath M |

#### Project Tracker, Velocity & Burndown Chart: (4 Marks)

| Sprint   | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed (as on Planned End Date) | Sprint Release Date (Actual) |
|----------|--------------------|----------|-------------------|---------------------------|-------------------------------------------------|------------------------------|
| Sprint-1 | 20                 | 6 Days   | 24 Oct 2022       | 29 Oct 2022               | 20                                              | 29 Oct 2022                  |
| Sprint-2 | 20                 | 6 Days   | 31 Oct 2022       | 05 Nov 2022               |                                                 |                              |
| Sprint-3 | 20                 | 6 Days   | 07 Nov 2022       | 12 Nov 2022               |                                                 |                              |
| Sprint-4 | 20                 | 6 Days   | 14 Nov 2022       | 19 Nov 2022               |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |

## 6. Project Development Phase:

## **What you'll learn**

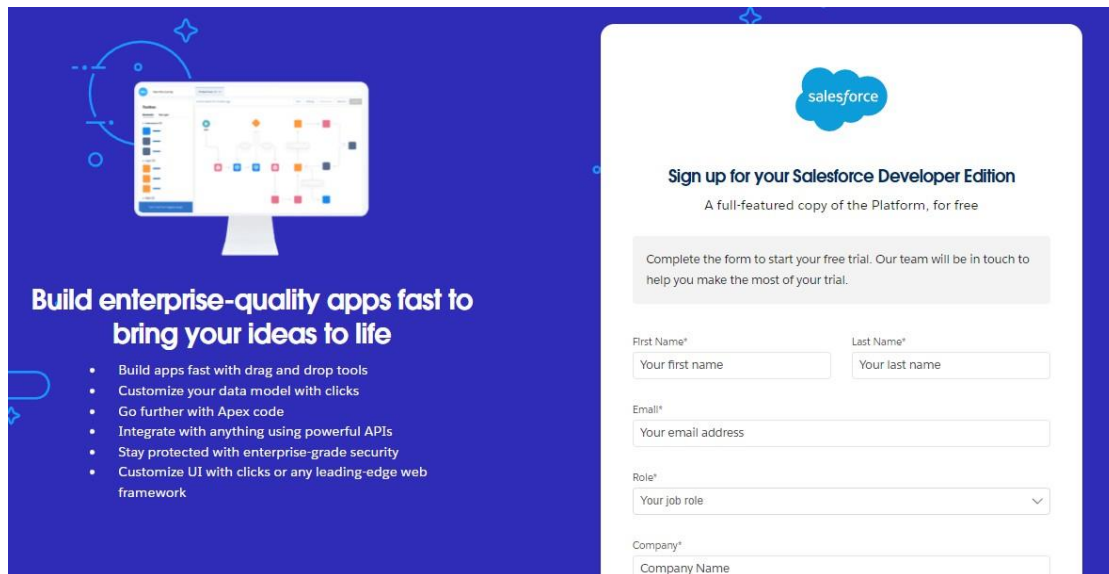
1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

## **Milestone 1: Salesforce Account**

### **Activity 1: Creating Developer Account**

Creating a developer org in salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. On the sign up form, enter the following details :



The image shows a promotional banner for Salesforce Developer Edition on the left and a sign-up form on the right. The banner features a blue background with a white monitor displaying a Salesforce interface. The text on the banner reads: "Build enterprise-quality apps fast to bring your ideas to life". Below this, there is a bulleted list of features: "Build apps fast with drag and drop tools", "Customize your data model with clicks", "Go further with Apex code", "Integrate with anything using powerful APIs", "Stay protected with enterprise-grade security", and "Customize UI with clicks or any leading-edge web framework". The sign-up form on the right has a white background with the Salesforce logo at the top. The heading is "Sign up for your Salesforce Developer Edition" with a subtext "A full-featured copy of the Platform, for free". Below this, there is a grey box with the text "Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial." The form fields are: "First Name\*" (with placeholder "Your first name"), "Last Name\*" (with placeholder "Your last name"), "Email\*" (with placeholder "Your email address"), "Role\*" (a dropdown menu with "Your job role" and a downward arrow), and "Company\*" (with placeholder "Company Name").

**Build enterprise-quality apps fast to bring your ideas to life**

- Build apps fast with drag and drop tools
- Customize your data model with clicks
- Go further with Apex code
- Integrate with anything using powerful APIs
- Stay protected with enterprise-grade security
- Customize UI with clicks or any leading-edge web framework

**Sign up for your Salesforce Developer Edition**  
A full-featured copy of the Platform, for free

Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial.

First Name\*  
Your first name

Last Name\*  
Your last name

Email\*  
Your email address

Role\*  
Your job role

Company\*  
Company Name

1) First name & Last name

2) Email

3) Role : Developer

4) Company : College Name

5) County : India

6) Postal Code : pin code

7) Username : should be a combination of your name and company

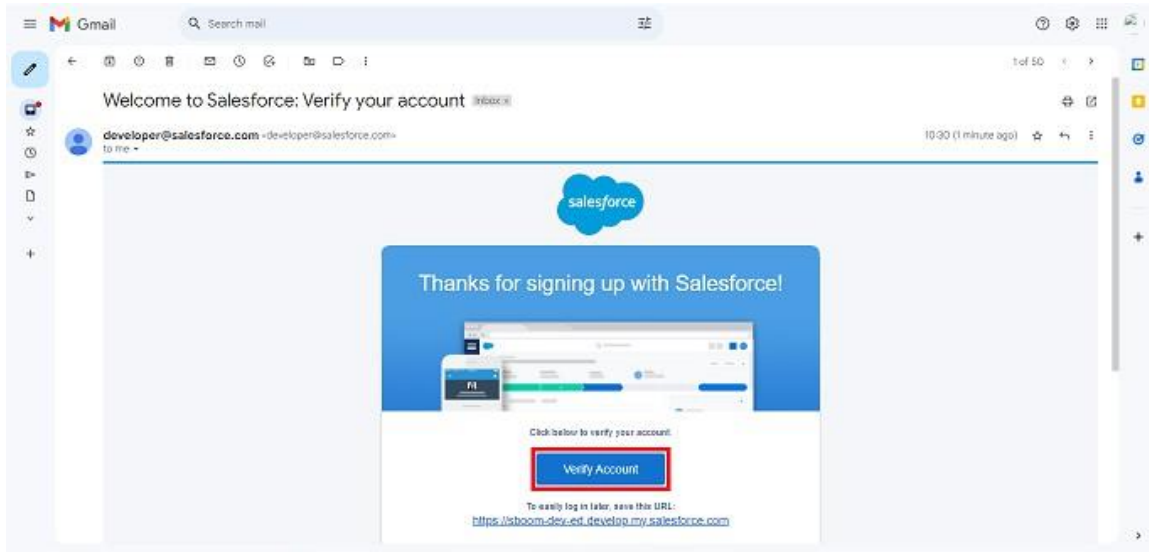
This need not be an actual email id, you can give anything in the format :

[username@organization.com](mailto:username@organization.com)

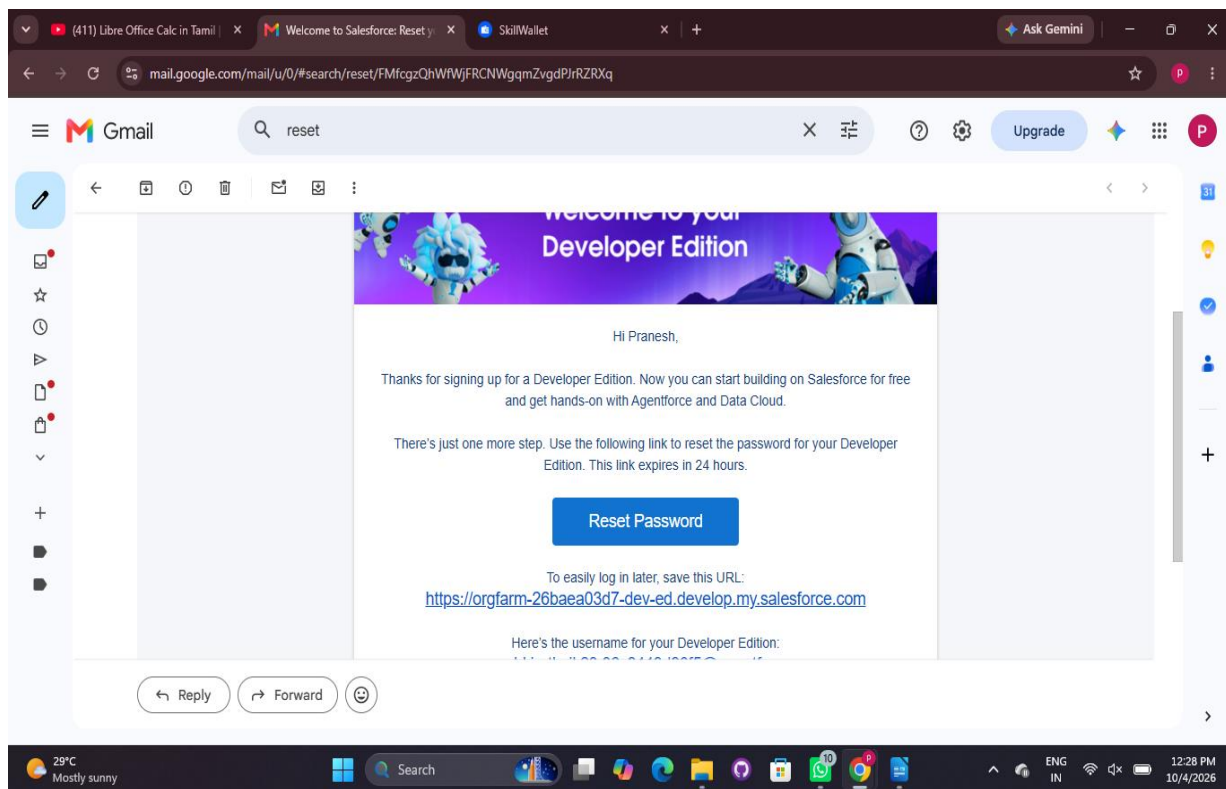
Click on sign me up after filling these.

## Activity 2: Account Activation

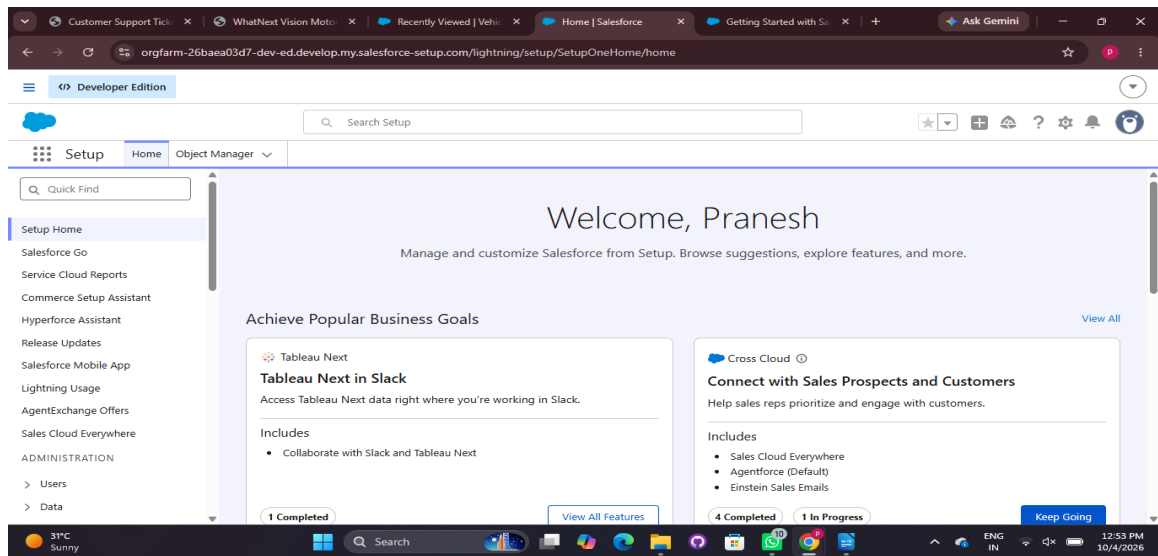
1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.



4. Then you will redirect to your salesforce setup page.



## Milestone 2: Data Management-Objects

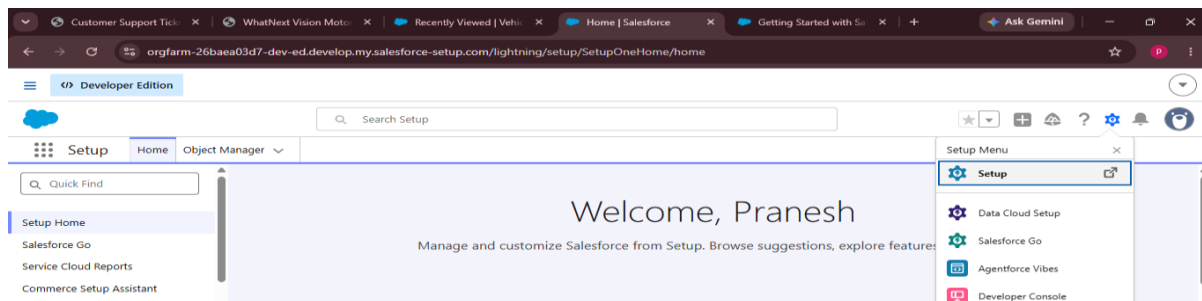
### Objects & Relationships

| Object Name                | Purpose                       | Relationships                   |
|----------------------------|-------------------------------|---------------------------------|
| <b>Vehicle__c</b>          | Stores vehicle details        | Related to Dealer & Orders      |
| <b>Vehicle_Dealer__c</b>   | Stores authorized dealer info | Related to Orders               |
| <b>Vehicle_Customer__c</b> | Stores customer details       | Related to Orders & Test Drives |
| <b>Vehicle_Order__c</b>    | Tracks vehicle purchases      | Related to Customer & Vehicle   |

|                                   |                                   |                               |
|-----------------------------------|-----------------------------------|-------------------------------|
| <b>Vehicle_Test_Drive__c</b>      | Tracks test drive bookings        | Related to Customer & Vehicle |
| <b>Vehicle_Service_Request__c</b> | Tracks vehicle servicing requests | Related to Customer & Vehicle |

## To Navigate to Setup page:

Click on gear icon → click setup.

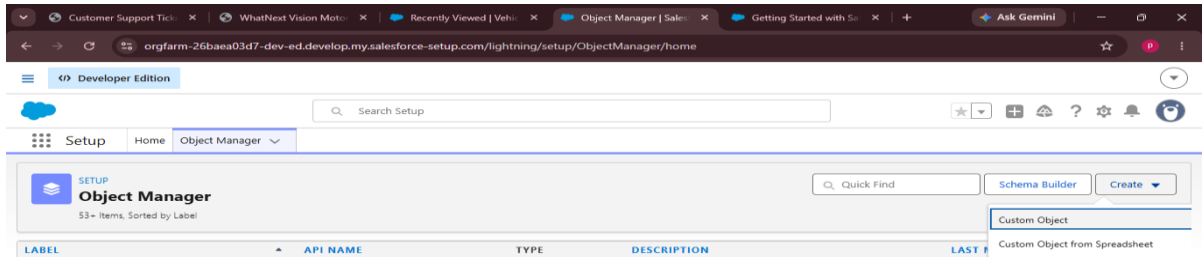


## Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



- 1) Enter the label name → Vehicle
- 2) Plural label name → Vehicles
- 3) Enter Record Name Label and Format
  - Record Name → Vehicle Name
  - Data Type → Text

**Custom Object Definition Edit** [Save] [Save & New] [Cancel]

**Custom Object Information** ! Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label:  Example: Account

Plural Label:  Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name:  Example: Account

Description:

Context-Sensitive Help Setting

☒ Open the standard Salesforce.com Help & Training window

☐ Open a window using a Visualforce page

2. Click on Allow reports,
3. Allow search → **Save.**

☒ Allow Sharing  
☒ Allow Bulk API Access  
☒ Allow Streaming API Access

**Deployment Status** [What is this?](#)  
☐ In Development  
☒ Deployed

**Search Status**  
 When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)  
☐ Allow Search

**Object Creation Options (Available only when custom object is first created)**  
☐ Add Notes and Attachments related list to default page layout  
☐ Launch New Custom Tab Wizard after saving this custom object

## Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
  - 1) Enter the label name→ Vehicle Dealer
  - 2) Plural label name→ Vehicle Dealers
  - 3) Enter Record Name Label and Format
    - Record Name → Dealer Name
    - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

## Activity 3: Creating remaining objects

Creating remaining objects

Create remaining objects which have mentioned in the above table Data Management-Objects

## Milestone 3: Data Management-Tabs

### Activity 1: Creating a Custom Tab(Vehicle)

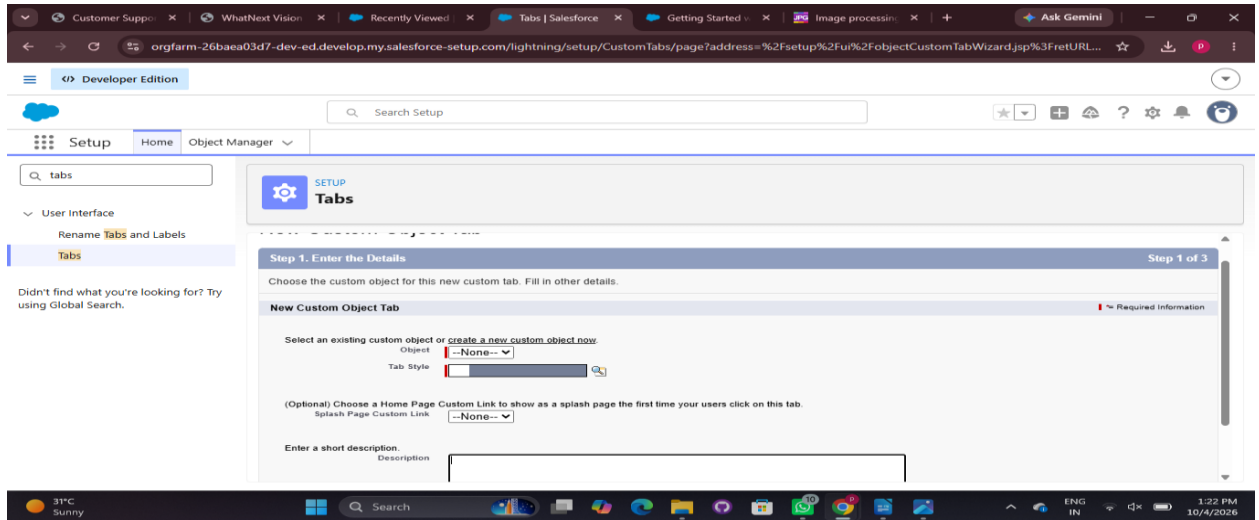
1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'tabs' entered, and the 'User Interface' section is expanded, showing 'Rename Tabs and Labels' and 'Tabs' (selected). The main content area is titled 'SETUP Tabs' and includes a description: 'Visualforce tabs allow you to embed Visualforce pages. Lightning Component tabs allow you to add Lightning components to the navigation menu in Lightning Experience and the mobile app. Lightning Page tabs allow you to add Lightning Pages to Lightning Experience and the mobile app.' Below this, there are two sections: 'Custom Object Tabs' and 'Web Tabs'. The 'Custom Object Tabs' section has a table with the following data:

| Action                                     | Label                                    | Tab Style  | Description |
|--------------------------------------------|------------------------------------------|------------|-------------|
| <a href="#">Edit</a>   <a href="#">Del</a> | <a href="#">Vehicle Customers</a>        | People     |             |
| <a href="#">Edit</a>   <a href="#">Del</a> | <a href="#">Vehicle Dealers</a>          | Car        |             |
| <a href="#">Edit</a>   <a href="#">Del</a> | <a href="#">Vehicle Orders</a>           | Boat       |             |
| <a href="#">Edit</a>   <a href="#">Del</a> | <a href="#">Vehicles</a>                 | Airplane   |             |
| <a href="#">Edit</a>   <a href="#">Del</a> | <a href="#">Vehicle Service Requests</a> | Camera     |             |
| <a href="#">Edit</a>   <a href="#">Del</a> | <a href="#">Vehicle Test Drives</a>      | Helicopter |             |

The 'Web Tabs' section below it shows 'No Web Tabs have been defined'.

2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



## Activity 2:

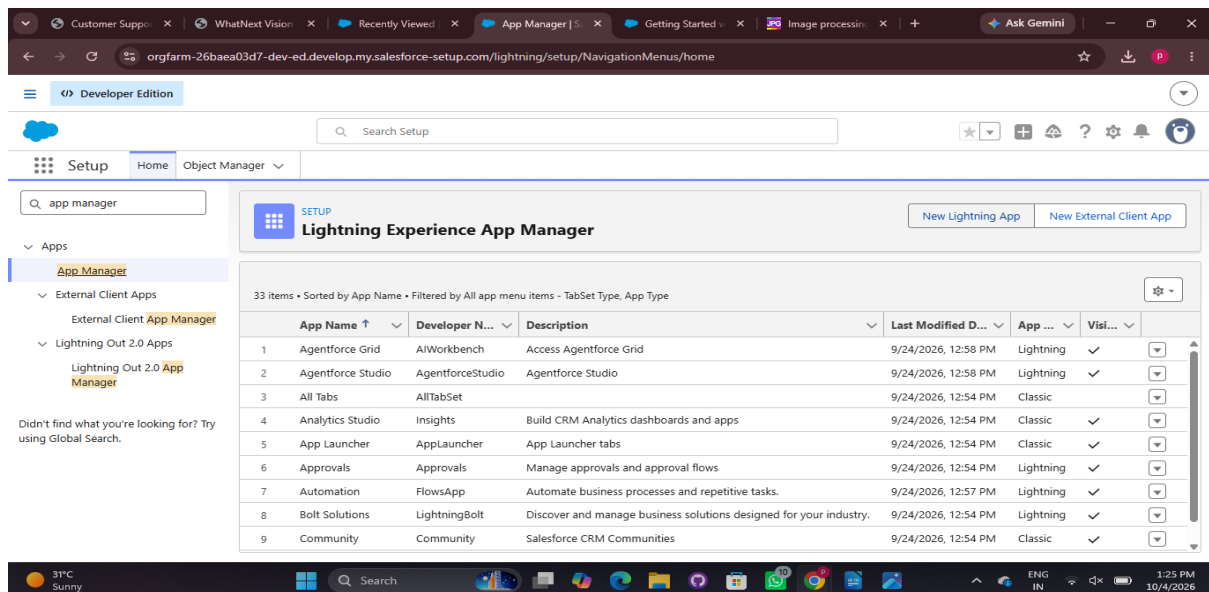
Now create tabs for Other objects as well which we have created above Objects.

## Milestone 4: Data Management-App Manager

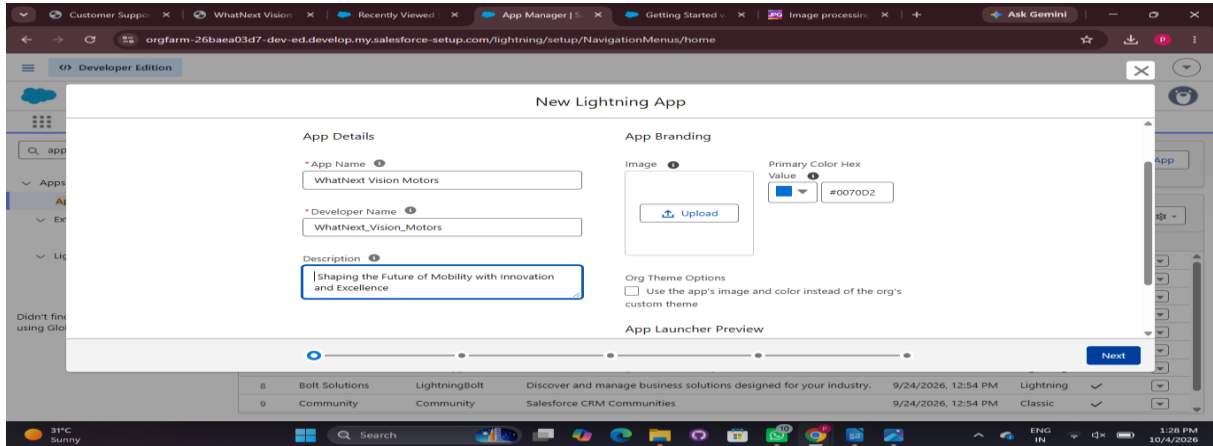
### Activity 1: Create a Lightning App

To create a lightning app page:

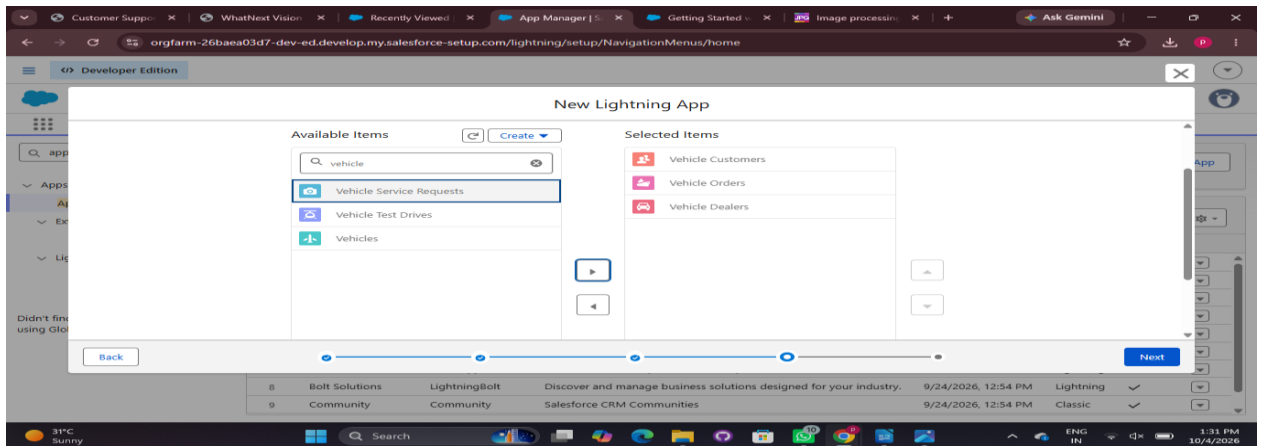
1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



2. Fill the app name in app details and branding as follow  
App Name : WhatNext Vision Motors  
Developer Name : this will auto populated  
Description : Give a meaningful description  
Image : optional (if you want to give any image you can otherwise not mandatory)  
Primary color hex value : keep this default
3. Then click Next → (App option page) keep it as default → Next → (Utility Items)  
keep it as default → Next.



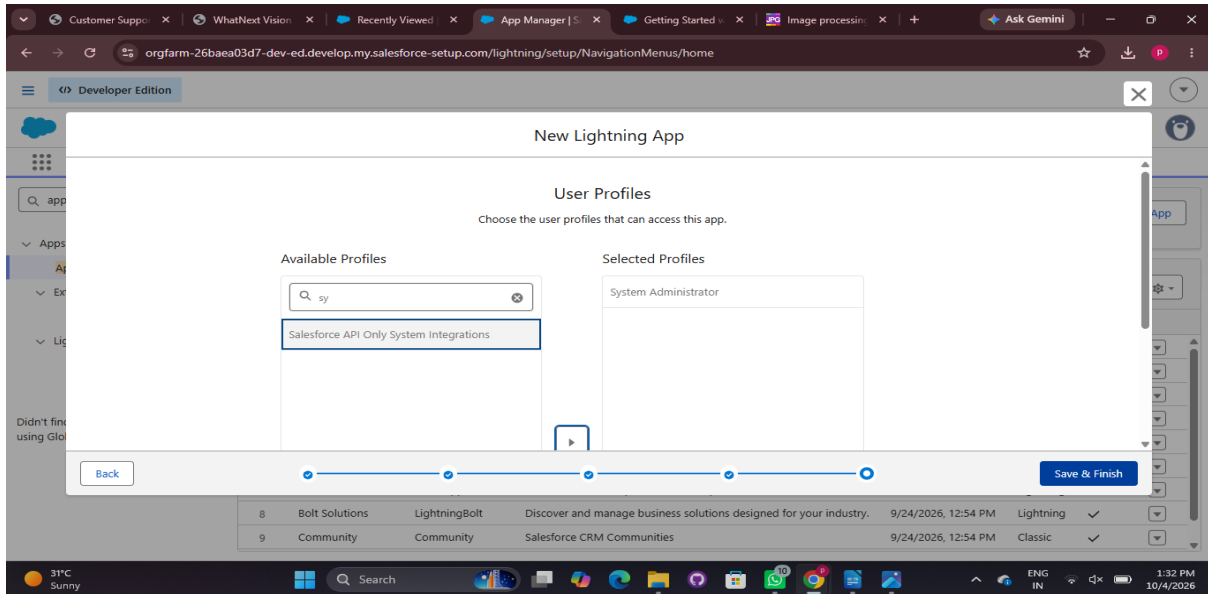
4. To Add Navigation Items:



Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

**Note:** select the custom object which we have created in the previous activity.

5. To Add User Profiles:



Search profiles (System administrator) in the search bar → click on the arrow button  
→ save & finish.

## Milestone 5: Data Management-Fields

### Key Fields for Each Object

#### 1. Vehicle\_\_c (Custom Object)

- **Vehicle\_Name\_\_c** (Text)
- **Vehicle\_Model\_\_c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock\_Quantity\_\_c** (Number)
- **Price\_\_c** (Currency)
- **Dealer\_\_c** (Lookup to Dealer\_\_c)
- **Status\_\_c** (Picklist: Available, Out of Stock, Discontinued)

#### 2. Vehicle\_Dealer\_\_c (Custom Object)

- **Dealer\_Name\_\_c** (Text)
- **Dealer\_Location\_\_c** (Text)
- **Dealer\_Code\_\_c** (Auto Number)
- **Phone\_\_c** (Phone)
- **Email\_\_c** (Email)

#### 3. Vehicle\_Order\_\_c (Custom Object)

- **Customer\_\_c** (Lookup to Customer\_\_c)
- **Vehicle\_\_c** (Lookup to Vehicle\_\_c)
- **Order\_Date\_\_c** (Date)
- **Status\_\_c** (Picklist: Pending, Confirmed, Delivered, Canceled)

#### 4. Vehicle\_Customer\_\_c (Custom Object)

- **Customer\_Name\_\_c** (Text)
- **Email\_\_c** (Email)
- **Phone\_\_c** (Phone)
- **Address\_\_c** (Text)
- **Preferred\_Vehicle\_Type\_\_c** (Picklist: Sedan, SUV, EV, etc.)

#### 5. Vehicle\_Test\_Drive\_\_c (Custom Object)

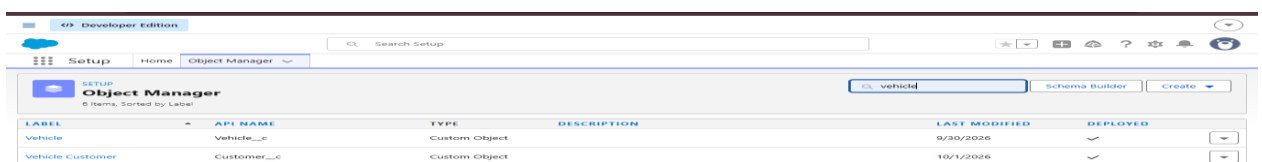
- **Customer\_\_c** (Lookup to Customer\_\_c)
- **Vehicle\_\_c** (Lookup to Vehicle\_\_c)
- **Test\_Drive\_Date\_\_c** (Date)
- **Status\_\_c** (Picklist: Scheduled, Completed, Canceled)

#### 6. Vehicle\_Service\_Request\_\_c (Custom Object)

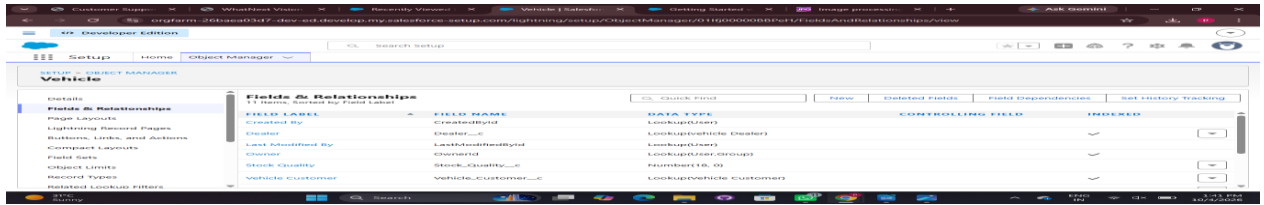
- **Customer\_\_c** (Lookup to Customer\_\_c)
- **Vehicle\_\_c** (Lookup to Vehicle\_\_c)
- **Service\_Date\_\_c** (Date)
- **Issue\_Description\_\_c** (Text)
- **Status\_\_c** (Picklist: Requested, In Progress, Completed)

### Object Field Creation :

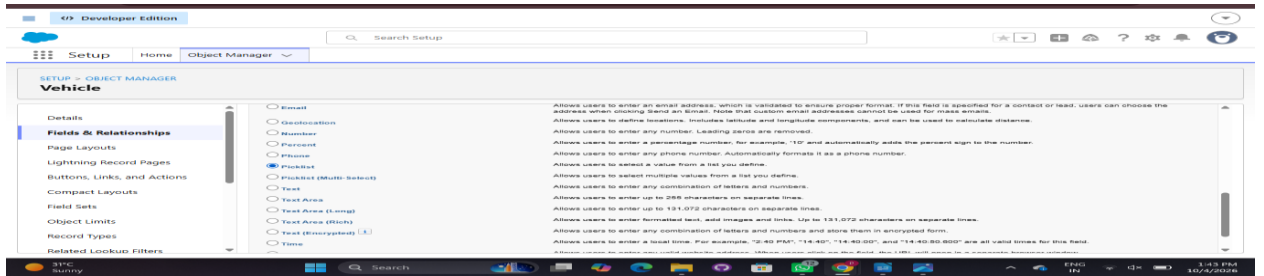
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



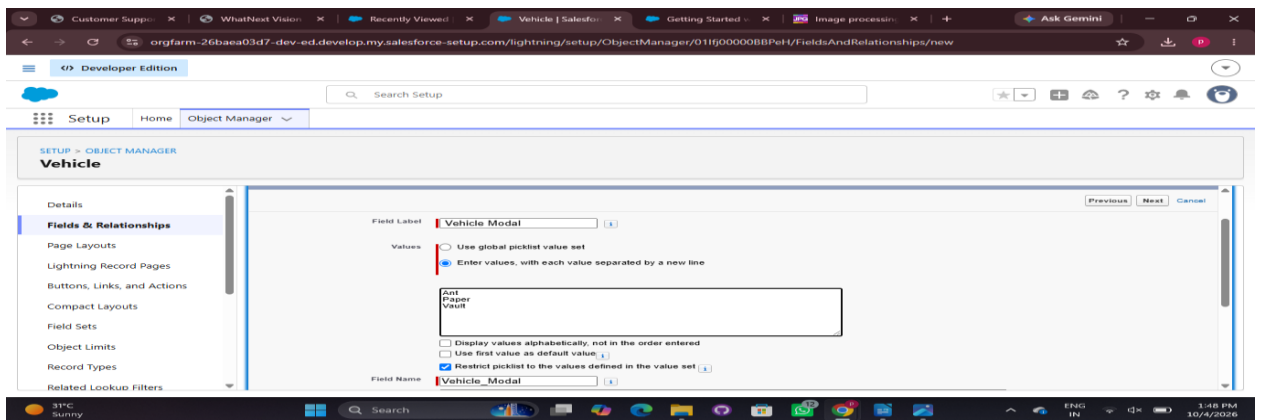
2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



4. Click on Next

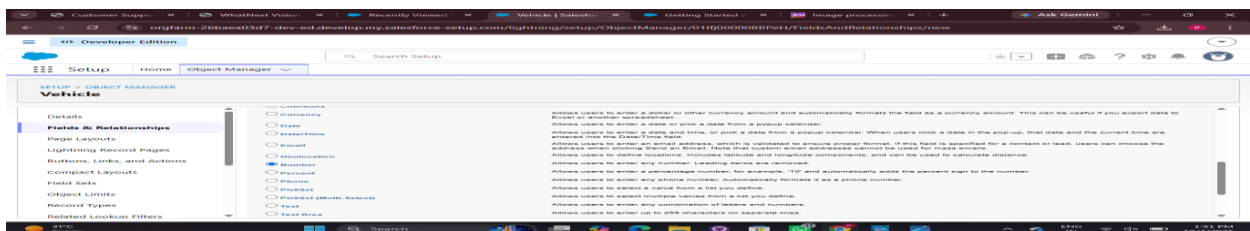


5. Fill the above as following:

- o Field Label: Vehicle Modal
- o Fill PickList Values
- o Click on Next → Next → Save and new.

## Object Field Creation-2 :

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.



### 3. Click on Next

Vehicle  
New Custom Field

Help for this Page

Step 2. Enter the details

Step 2 of 4

Field Label

Please enter the length of the number and the number of decimal places. For example, a number with a length of 8 and 2 decimal places can accept values up to "12345678.90".

Length  Number of digits to the left of the decimal point

Decimal Places  Number of digits to the right of the decimal point

Field Name

Description

Previous Next Cancel

### 4. Fill the above as following:

- Field Label: Stock Quality
- Click on Next → Next → Save and new.

## Relationship Field Creation :

### Creating a Lookup Relationship in Vehicle Object to Dealer Object.

#### 1. Select DataType as Lookup Relationship.

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Vehicle

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Data Type

Select one of the data types below.

☐ None Selected

☐ Auto Number

☐ Formula

☐ Roll-Up Summary

☒ Lookup Relationship

☐ Master-Detail Relationship

☐ External Lookup Relationship

☐ Checkbox

Creates a relationship that links this object to another object. The relationship field allows users to click on a lookup icon to select a value from a popup list. The other object is the source of the values in the list.

Creates a special type of parent-child relationship between this object (the child, or "detail") and another object (the parent, or "master") where:

- The relationship field is required on all detail records.
- The ownership and sharing of a detail record are determined by the master record.
- When a user deletes the master record, all detail records are deleted.
- You can create rollup summary fields on the master record to summarize the detail records.

The relationship field allows users to click on a lookup icon to select a value from a popup list. The master object is the source of the values in the list.

Creates a relationship that links this object to an external object whose data is stored outside the Salesforce org.

Allows users to select a True (checked) or False (unchecked) value.

#### 2. Click Next

#### 3. Select Dealer Object Form Related Field.

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Vehicle

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

New Relationship

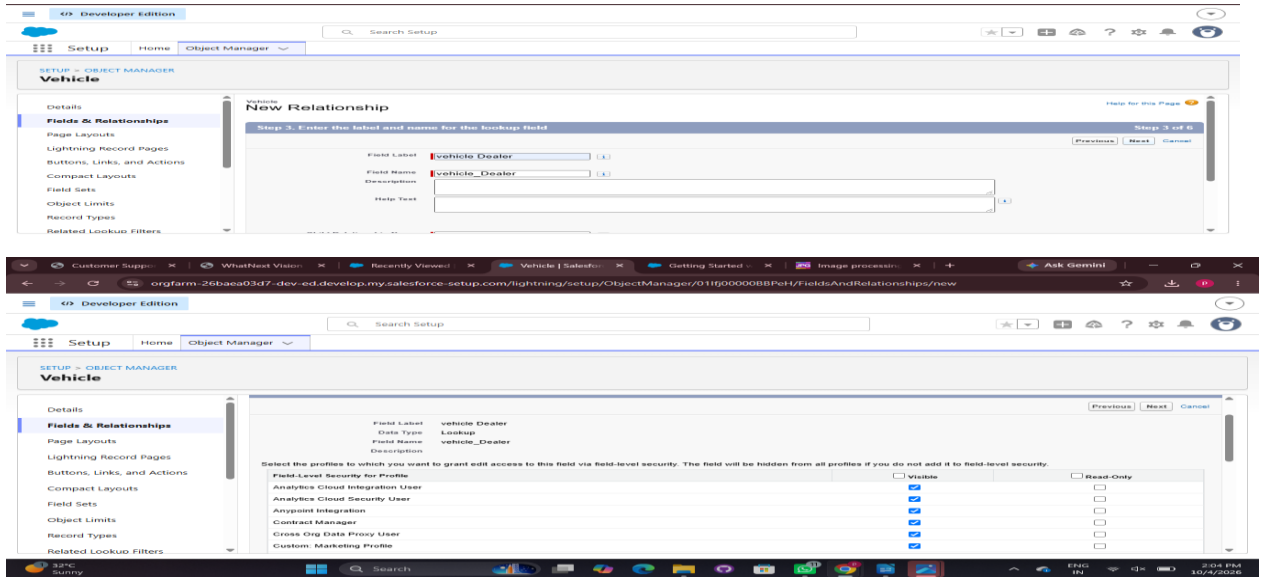
Step 2. Choose the related object

Step 2 of 2

Select the other object to which this object is related.

Related To

Previous Next Cancel



4. Click Next and Save.

## More Fields Creation

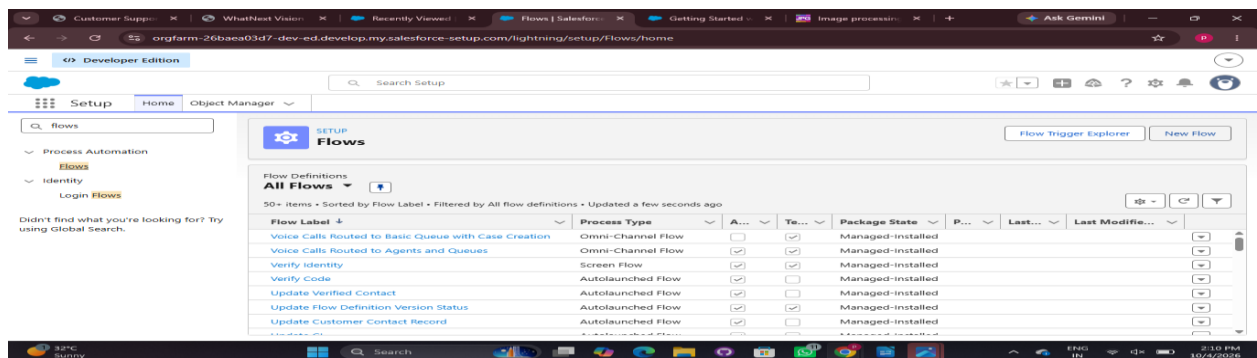
Create remaining fields as mentioned in the Data Management-Fields

## Milestone 6: Automation

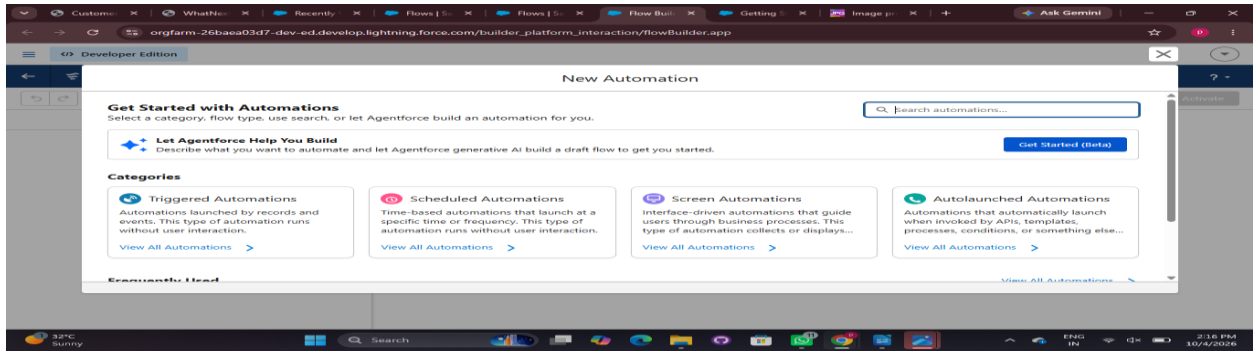
### Flow Creation :

**Creating a record triggered flow to assign nearest dealer to the customer's location**

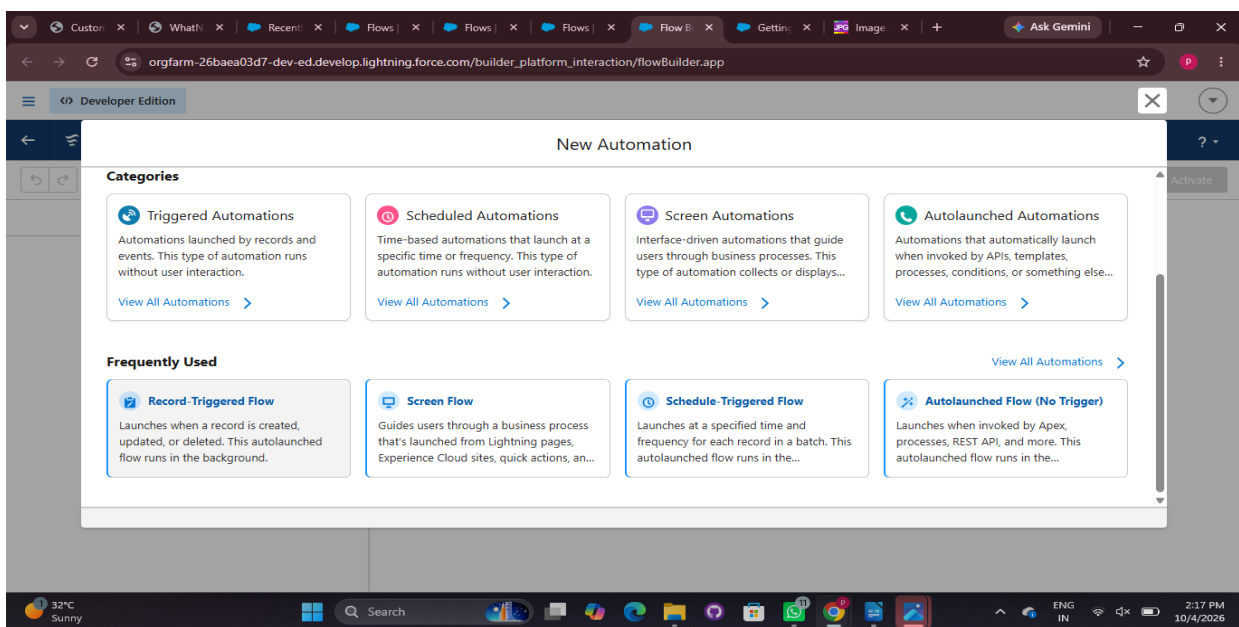
Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Start From Scratch and click Next.



Step 3 : Select Record-Triggered Flow and click Create



Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

**Set Entry Condition :**

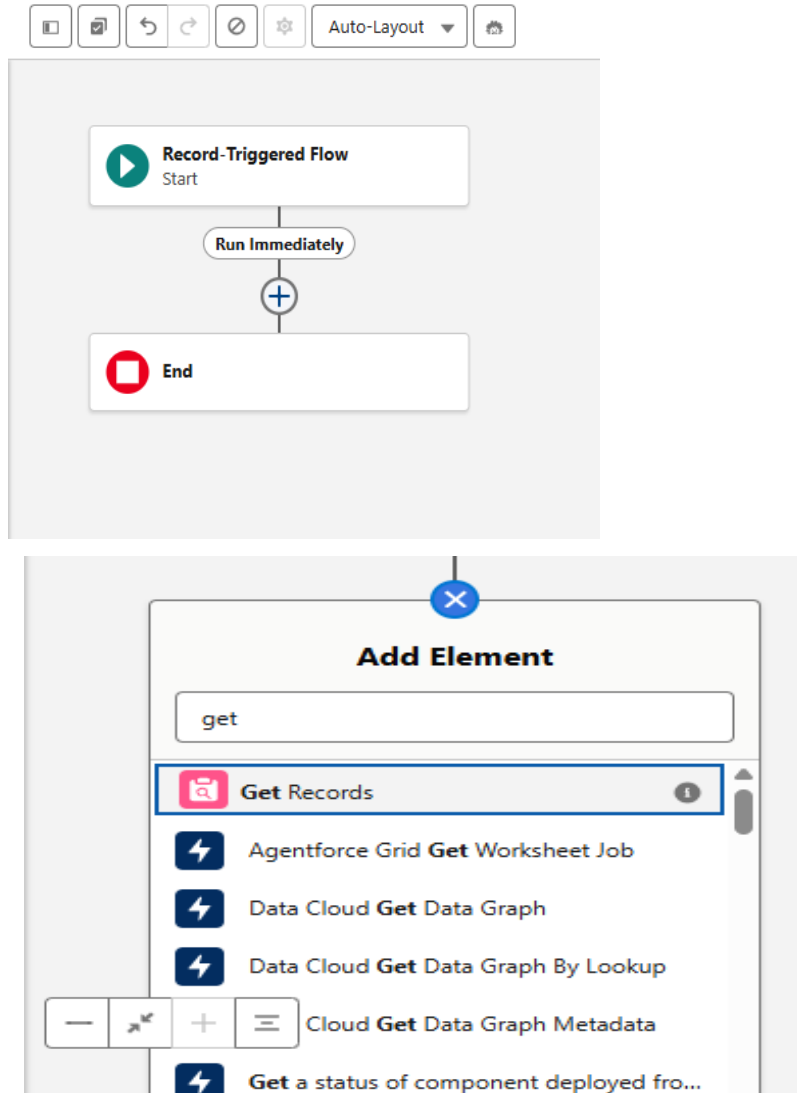
All Conditions Are Met (AND)

Filed : Status\_\_c

Operator : Equals

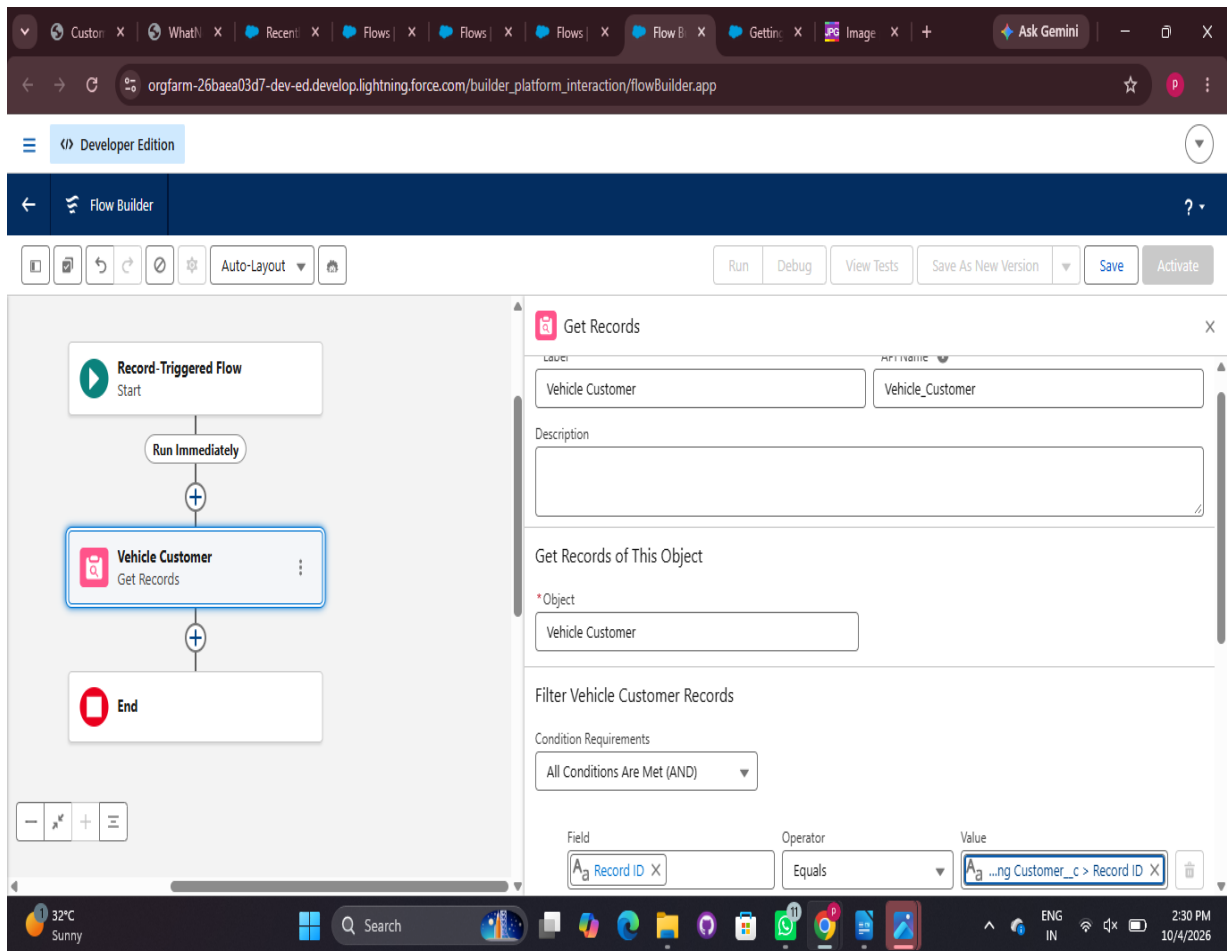
Value : Pending

Step 5 : Click + → Select **Get Records**



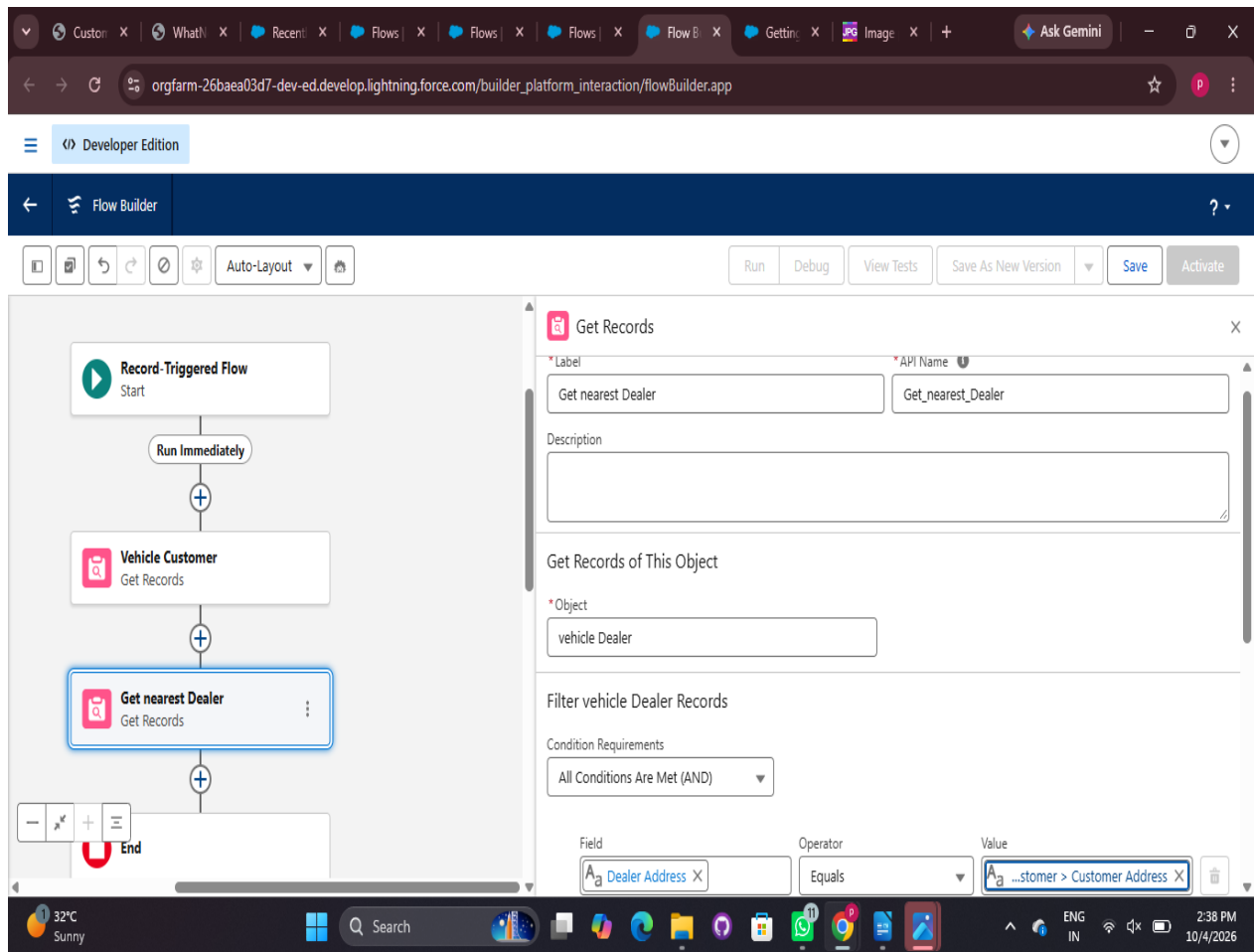
Step 6 : **Label** : Get Customer Information

- Object : Vehicle Customer
- Condition :
- Field : Id
- Operator : Equals
- Value : {!\$Record.Vehicle\_Customer\_\_c}



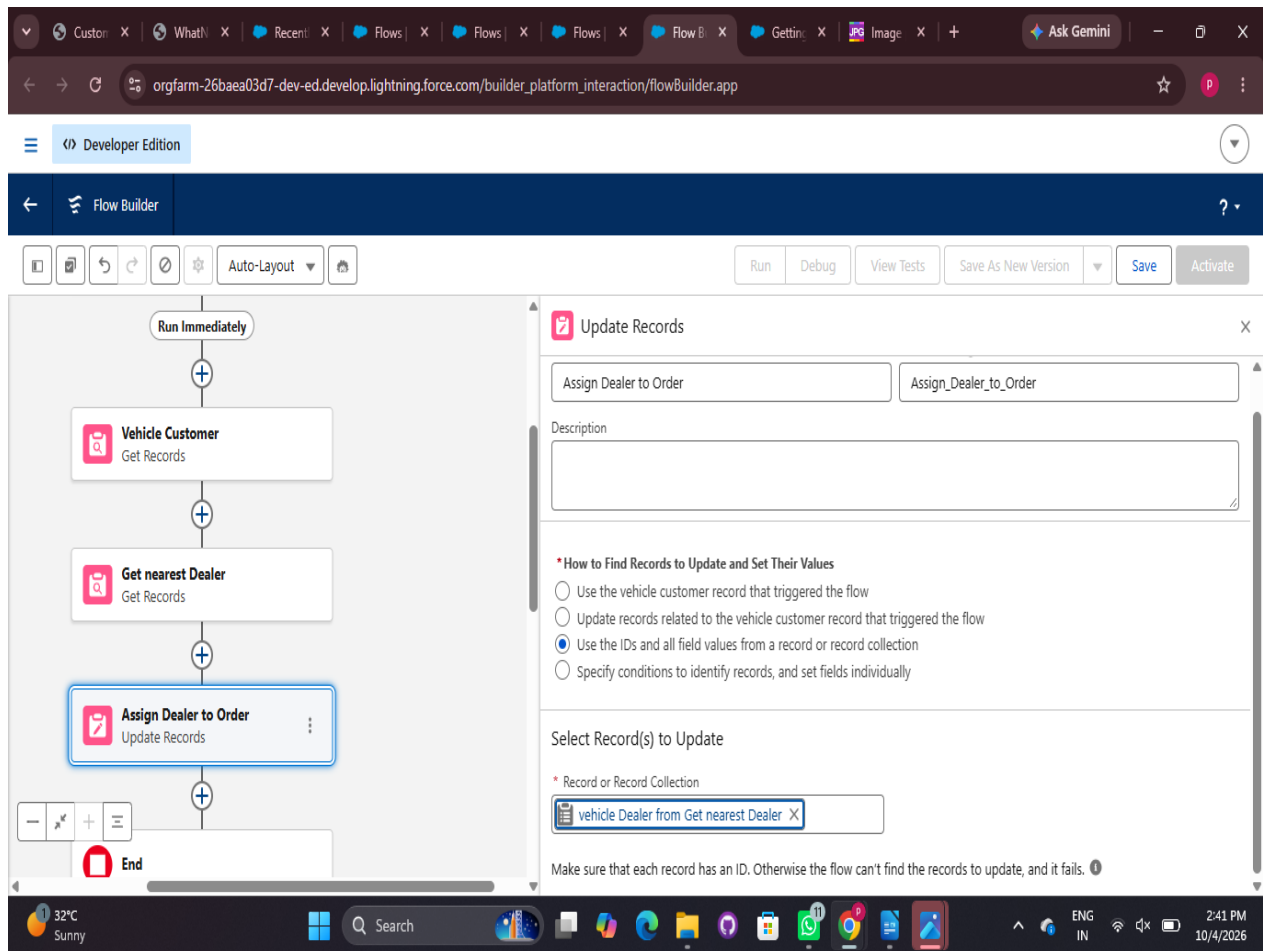
Step 7 : Click + → Select **Get Records**

- **Label** : Get nearest Dealer
- **Object** : Vehicle Dealer
- **Condition** :
- **Field** : Dealer\_Location\_\_c
- **Operator** : Equals
- **Value** : {!Get\_Customer\_Information.Address\_\_c}



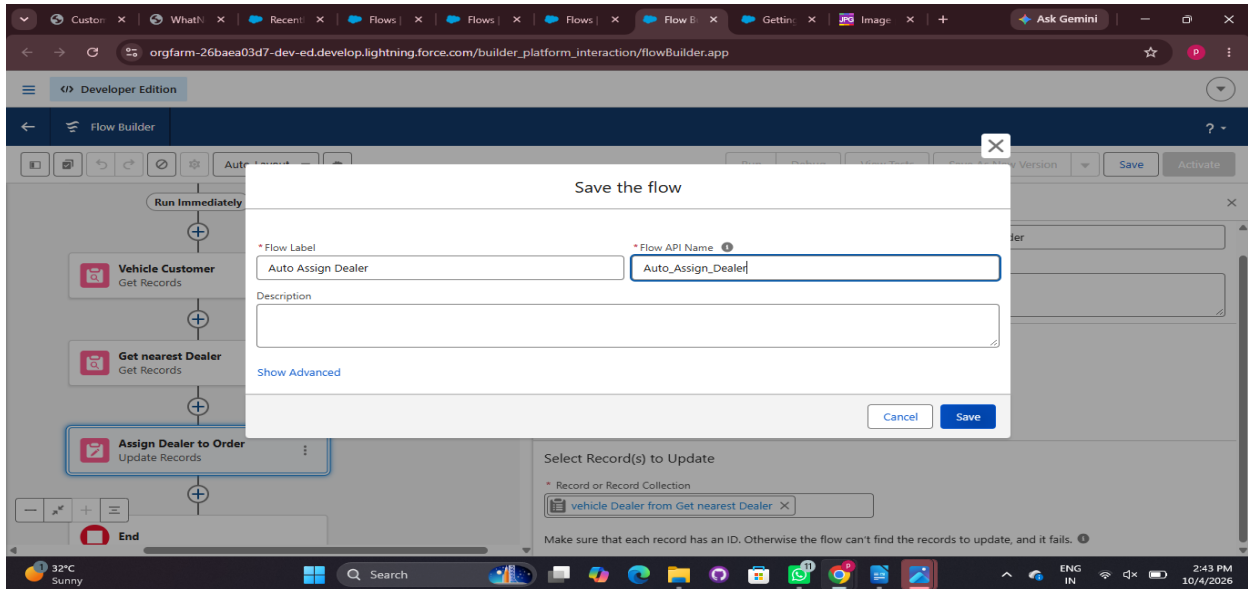
## Step 8 : Click + → Select **Update Records**

- **Label** : Assign Dealer to Order
- \*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection
- Select Record(s) to Update : {!Get\_Nearest\_Dealer}

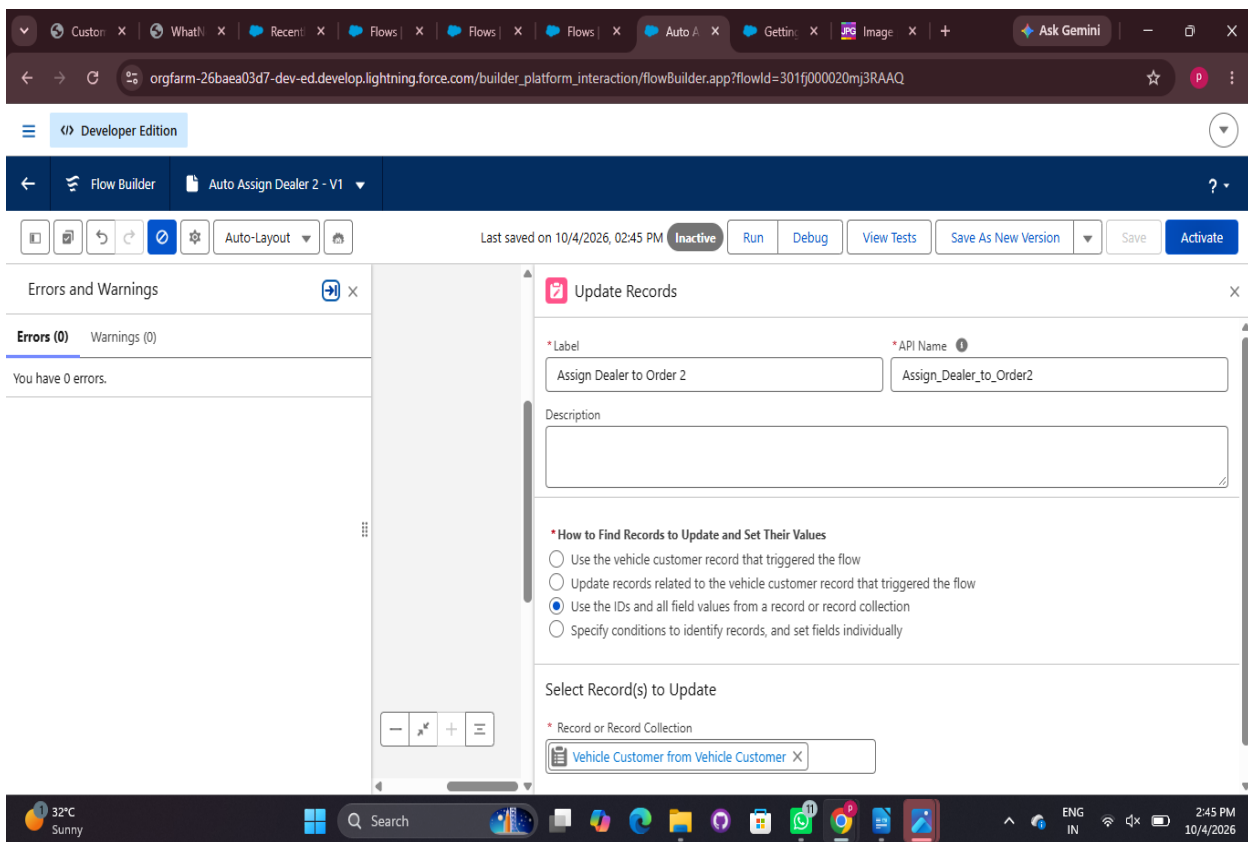


**Step 9 :** Click Save and Give label Name and Activate Flow.

- Label Name : Auto Assign Dealer



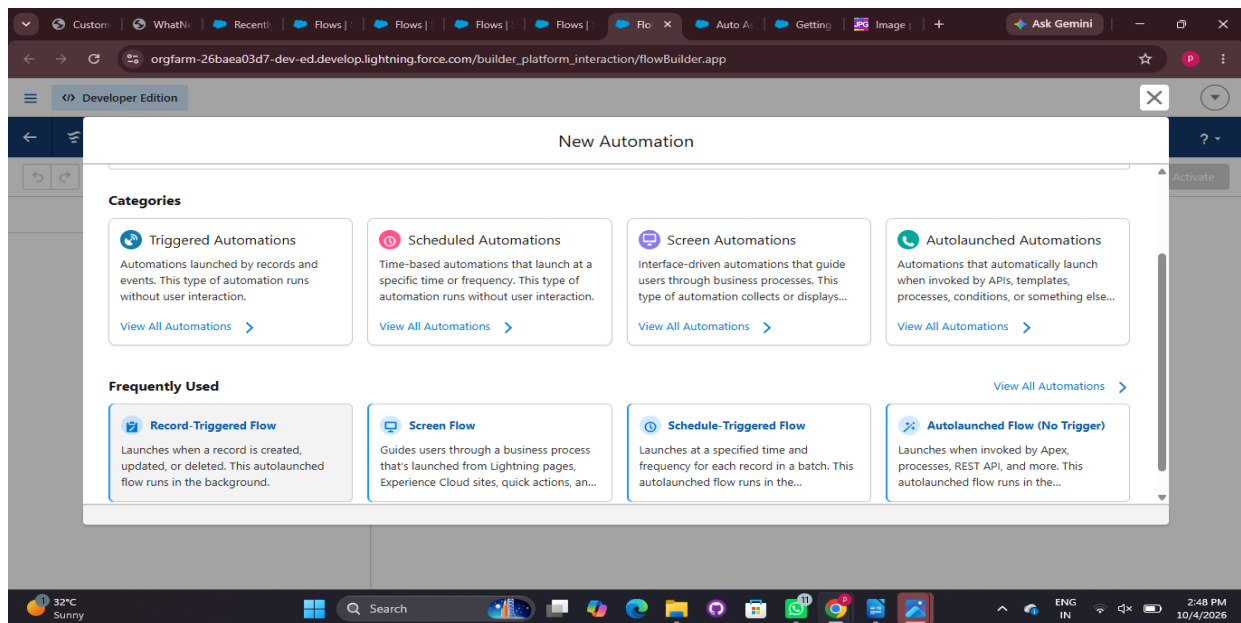
## Step 10 : Activate Flow



## Record-Trigger Flow Creation :

**Creating a record triggered flow to send an email to the customer reminding about the test drive.**

**Step 1 :** Select Record-Triggered Flow and click Create



**Step 2 :** Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

**Set Entry Condition :**

- All Conditions Are Met (AND)
- Filed : Status\_\_c
- Operator : Equals
- Value : Scheduled

**Step 3 :** Click + Add Scheduled Paths (below the trigger).

- Label: Reminder Before Test Drive.
- Time Source: Test\_Drive\_Date\_\_c
- Offset Number: 1.
- Offset Options: Days Before.

- Click Done.

The image shows a screenshot of the Salesforce Flow Builder interface. At the top, a modal window titled "Record-Triggered Flow" is displayed, showing the configuration for a flow named "Vehicle Test Drive". The configuration includes:

- Object:** Vehicle Test Drive
- Trigger:** A record is created or updated
- Conditions:** 1
- Optimize for:** Actions and Related Records
- Scheduled Paths:** 2

Below the modal, the main Flow Builder canvas is visible, showing a flow diagram with a "Run Immediately" path and a "Reminder Before Test Drive" path. The "Configure Scheduled Paths" dialog is open on the right, showing the configuration for the "Reminder Before Test Drive" path:

- SCHEDULED PATHS:** 1
- Run Immediately:** Reminder Before Test Drive.
- SCHEDULED PATH DETAILS:**
  - \* Path Label:** Reminder Before Test Drive
  - \* API Name:** Reminder\_Before\_Test\_Drive
  - \* Time Source:** Test\_Drive\_\_c: Created Date
  - \* Offset Number:** 1
  - \* Offset Options:** Days Before

The bottom of the image shows the Windows taskbar with the date and time: 2:55 PM, 10/4/2026.

Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle\_Customer\_\_c.
4. Filter Conditions: Id = {!\$Record.Customer\_\_c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

The screenshot displays the Salesforce Flow Builder interface for a flow named "Test Drive Reminder - V1". The flow is in "Developer Edition" mode. The main canvas shows a flow diagram starting with a "Run Immediately" trigger, followed by a "Get Customer Information" action (labeled "Get Records"). Below this is a "Send Test Drive Reminder" action (labeled "Action"), and the flow ends with an "End" node. The right-hand pane provides configuration details for the "Get Records" action:

- Object:** Vehicle Test Drive
- Trigger:** A record is created or updated
- Optimize for:** Actions and Related Records
- Scheduled Paths:** 2
- Get Records of This Object:** Vehicle Customer
- Filter Vehicle Customer Records:**
  - Condition Requirements:** All Conditions Are Met (AND)
  - Field:** Created By ID
  - Operator:** Equals
  - Value:** ...\_Drive\_\_c > Test Drive Name

The bottom of the screen shows a Windows taskbar with the date and time as 2:57 PM on 10/4/2026.

## Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get\_Customer\_Information.Email\_\_c}
6. Rich-Text-Formatted Body : True

The screenshot displays the Salesforce Flow Builder interface. The top navigation bar shows the 'Developer Edition' tab. The main header indicates the flow is named 'Test Drive Reminder - V1'. The flow is currently in 'Active' status, with buttons for 'Run', 'Debug', 'View Tests', 'Save As New Version', 'Save', and 'Deactivate'. The flow diagram shows a trigger 'Object: Vehicle Test Drive' with the condition 'A record is created or updated'. This trigger branches into two paths: 'Run Immediately' and 'Reminder Before Test Dri...'. The 'Run Immediately' path leads to a 'Get Customer Information' action (Get Records), which then leads to a 'Send Test Drive Reminder' action (Send Email). The 'Reminder Before Test Dri...' path leads to an 'End' action. The right-hand pane shows the configuration for the 'Send Email' action, including fields for 'Label' (Send Test Drive Reminder), 'API Name' (Send\_Test\_Drive\_Reminder), and 'Description'. The 'Configure Recipient Details' section is expanded, showing a field for 'Recipient Addresses'.

7. Body : Create Variable

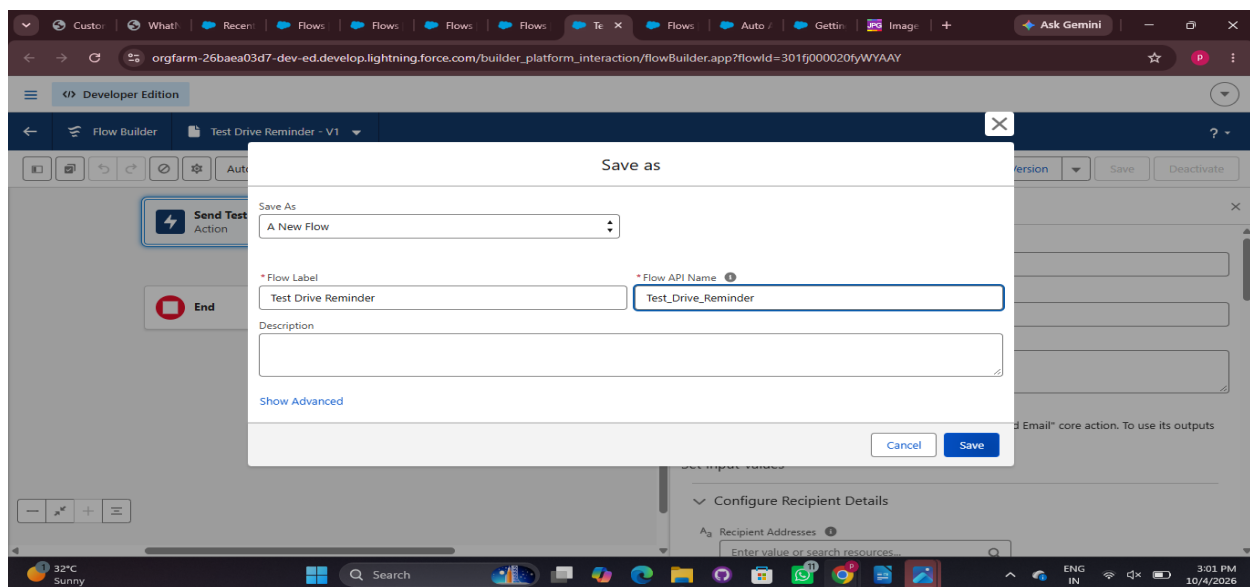
8 . Api Name : EmailSent

## Send Email

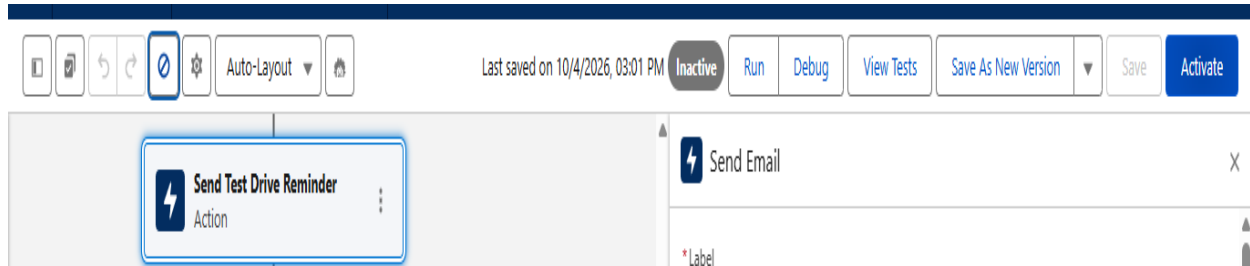
|                                                                                                                                                                                          |                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <b>A<sub>a</sub></b> Body                                                                               | <input checked="" type="checkbox"/> Included |
| <div> EmailSent </div> |                                              |
| <b>A<sub>a</sub></b> CC Recipient Address List                                                                                                                                           | <input type="checkbox"/> Not Included        |
| <b>A<sub>a</sub></b> Email Template ID                                                                                                                                                   | <input type="checkbox"/> Not Included        |
|  Log Email on Send                                                                                      | <input type="checkbox"/> Not Included        |
| <b>A<sub>a</sub></b> Recipient Address Collection                                                                                                                                        | <input type="checkbox"/> Not Included        |

Step 6 : Click save

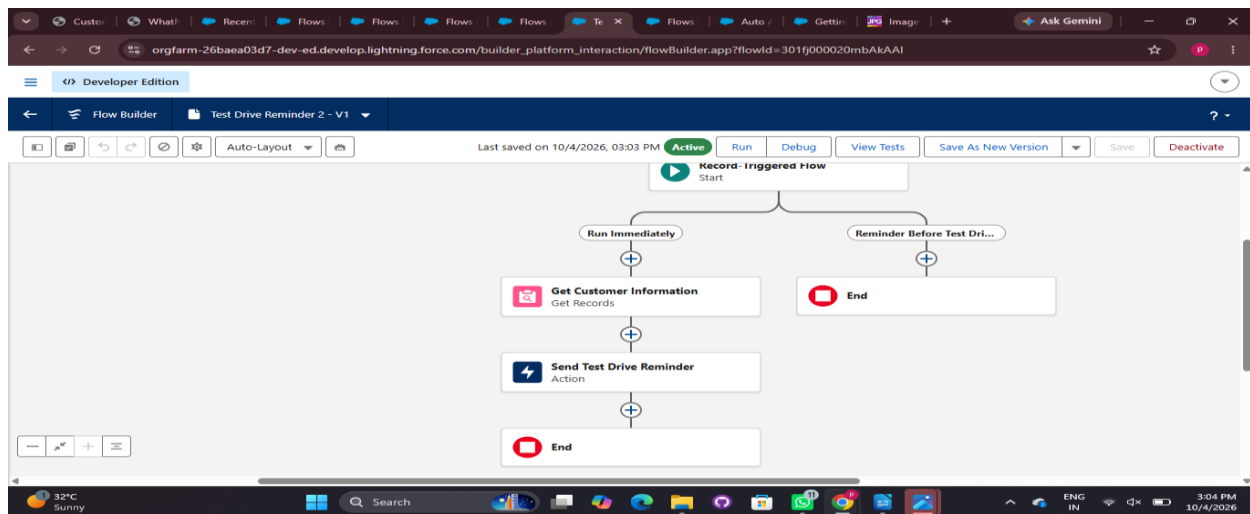
- **Label Name :** Test Drive Reminder



Step 7 : Activate Flow



## Flow Page After Creation :



## Test your Flow :

-  **Inbox** 76,244

 Starred

 Snoozed

 Important

 Sent

 Drafts

 Categories

 **Social** 3,451

 **Updates** 83,750

 Forums

Be careful with this message.

This may be a spoofed message. The message claims to have been sent from your account, but Gmail couldn't verify the actual source. Avoid clicking links or replying with sensitive information, unless you are sure that you actually sent this message. (No need to reset your password, the real sender does not actually have access to your account.)

Report spam

Looks safe



Dear test KHIT 123434,

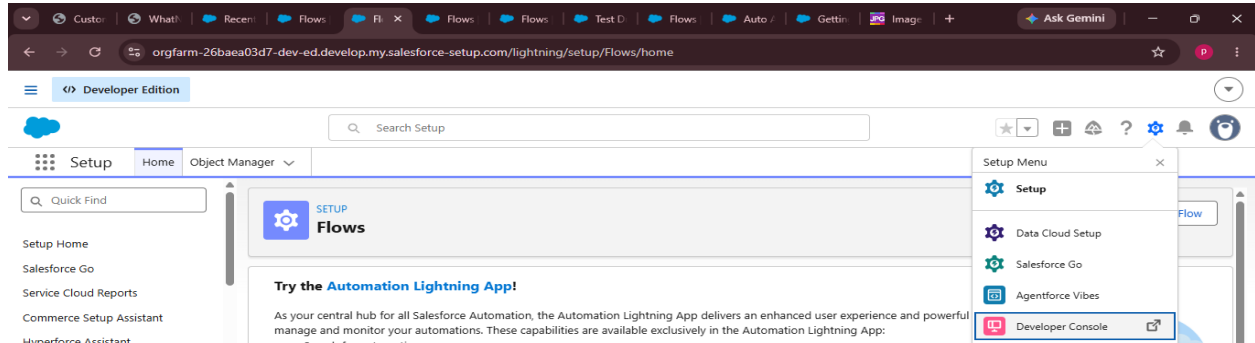
This is a reminder that your test drive for a08dM00000D0ck9QAB is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email].  
Thank you, WhatNext Vision Motors Team"



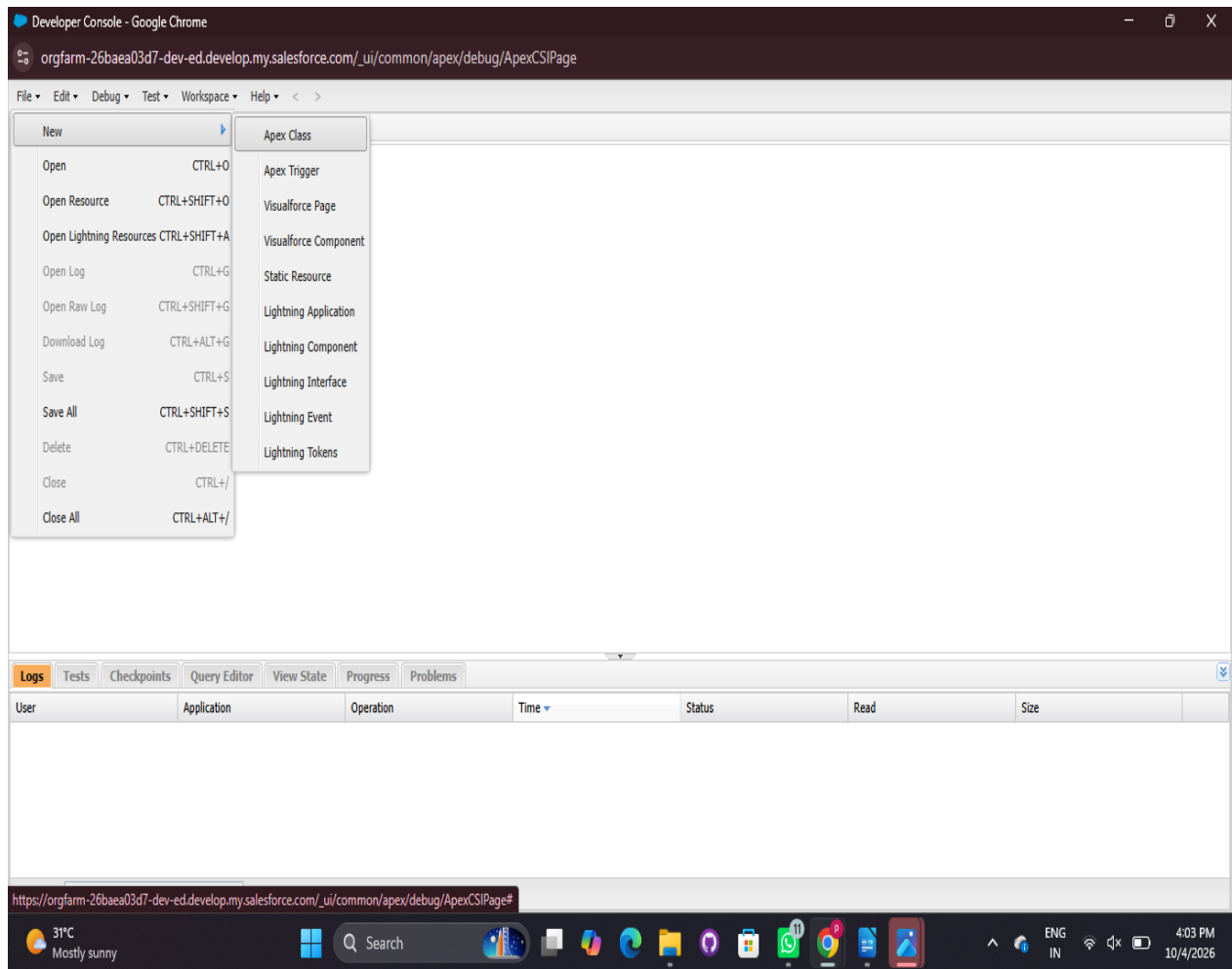
## Milestone 7: Apex and Batch Class

### Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

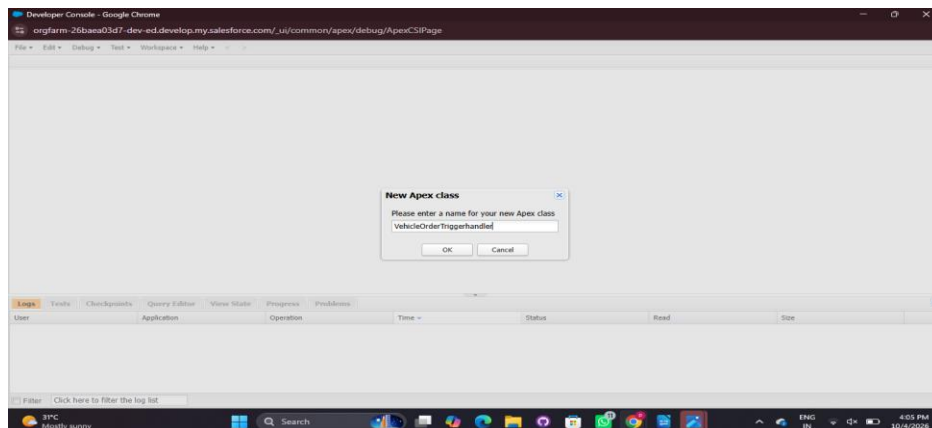


Step 2 : click File then Select New then Select Apex.



### Step 3 : Give Name For Apex Class

- Class Name : VehicleOrderTriggerhandler



### Step 4 : Write Apex Code

```
File • Edit • Debug • Test • Workspace • Help • < >
OrderTriggerHandler.apex • OrderTrigger.apex • InventoryBatchJob.apex • VehicleOrderTriggerHandler.apex • VehicleOrderTrigger.apex
Code Coverage: None • API Version: 52
Go To

1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_order__c> newOrders, Map<Id, Vehicle_order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
4         if (isBefore) {
5             if (isInsert || isUpdate) {
6                 preventOrderIfOutOfStock(newOrders);
7             }
8         }
9
10        if (isAfter) {
11            if (isInsert || isUpdate) {
12                updateStockOnOrderPlacement(newOrders);
13            }
14        }
15    }
16
17    // Method to prevent orders when the vehicle is out of stock
18    private static void preventOrderIfOutOfStock(List<Vehicle_order__c> orders) {
19        Set<Id> vehicleIds = new Set<Id>();
20        for (Vehicle_order__c order : orders) {
21            if (order.Vehicle__c != null) {
22                vehicleIds.add(order.Vehicle__c);
23            }
24        }
25
26        if (!vehicleIds.isEmpty()) {
27            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
28            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29                vehicleStockMap.put(vehicle.Id, vehicle);
30            }
31
32            for (Vehicle_order__c order : orders) {
33                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
34                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
35                    if (vehicle.Stock_Quantity__c <= 0) {
36                        order.addError('This vehicle is out of stock. Order cannot be placed.');

Logs Tests Checkpoints Query Editor View State Progress Problems



| User | Application | Operation | Time | Status | Read | Size |
|------|-------------|-----------|------|--------|------|------|
|------|-------------|-----------|------|--------|------|------|



Filter Click here to filter the log list


```

**Source Code :**

```
public class VehicleOrderTriggerHandler {
```

```
public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert,
Boolean isUpdate) {
```

```
    if (isBefore) {
```

```
        if (isInsert || isUpdate) {
```

```
            preventOrderIfOutOfStock(newOrders);
```

```
        }
```

```
    }
```

```
    if (isAfter) {
```

```
        if (isInsert || isUpdate) {
```

```
            updateStockOnOrderPlacement(newOrders);
```

```
        }
```

```
    }
```

```
}
```

```
// Method to prevent orders when the vehicle is out of stock
```

```
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
```

```
    Set<Id> vehicleIds = new Set<Id>();
```

```
    for (Vehicle_Order__c order : orders) {
```

```
        if (order.Vehicle__c != null) {
```

```
            vehicleIds.add(order.Vehicle__c);
```

```
        }
```

```
    }
```

```

if (!vehicleIds.isEmpty()) {

    Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();

    for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {

        vehicleStockMap.put(vehicle.Id, vehicle);

    }

    for (Vehicle_Order__c order : orders) {

        if (vehicleStockMap.containsKey(order.Vehicle__c)) {

            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);

            if (vehicle.Stock_Quantity__c <= 0) {

                order.addError('This vehicle is out of stock. Order cannot be placed.');
            }

        }

    }

}

// Method to update vehicle stock when an order is placed

private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {

    Set<Id> vehicleIds = new Set<Id>();

    for (Vehicle_Order__c order : orders) {

        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {

            vehicleIds.add(order.Vehicle__c);

        }

    }

}

```

```

    }

    if (!vehicleIds.isEmpty()) {

        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();

        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {

            vehicleStockMap.put(vehicle.Id, vehicle);

        }

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

        for (Vehicle_Order__c order : orders) {

            if (vehicleStockMap.containsKey(order.Vehicle__c)) {

                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);

                if (vehicle.Stock_Quantity__c > 0) {

                    vehicle.Stock_Quantity__c -= 1;

                    vehiclesToUpdate.add(vehicle);

                }

            }

        }

        if (!vehiclesToUpdate.isEmpty()) {

            update vehiclesToUpdate;

        }

    }

}

```

Vehicle Order  
VC-0003

\* = Required Information

Vehicle Order Name  
VC-0003

Owner  
Abdul Trainer

Vehicle Customer  
test KHIT 123434

Vehicle  
1erwe

Order Date  
21/02/2025

Status  
Pending

Created By  
Abdul Trainer, 13/02/2025, 5:34 am

By  
Abdul Trainer, 13/02/2025, 5:41 am

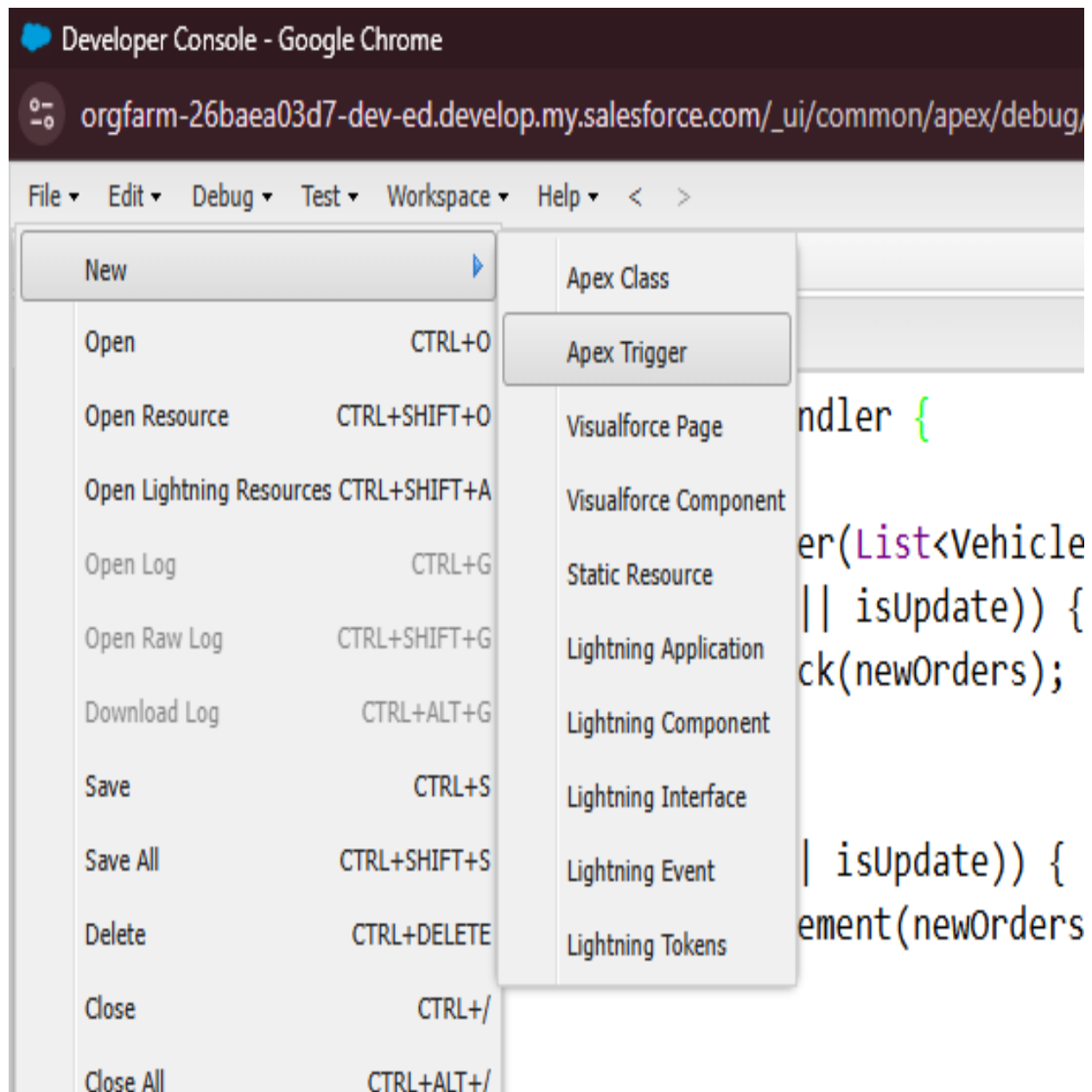
We hit a snag.

Review the errors on this page.

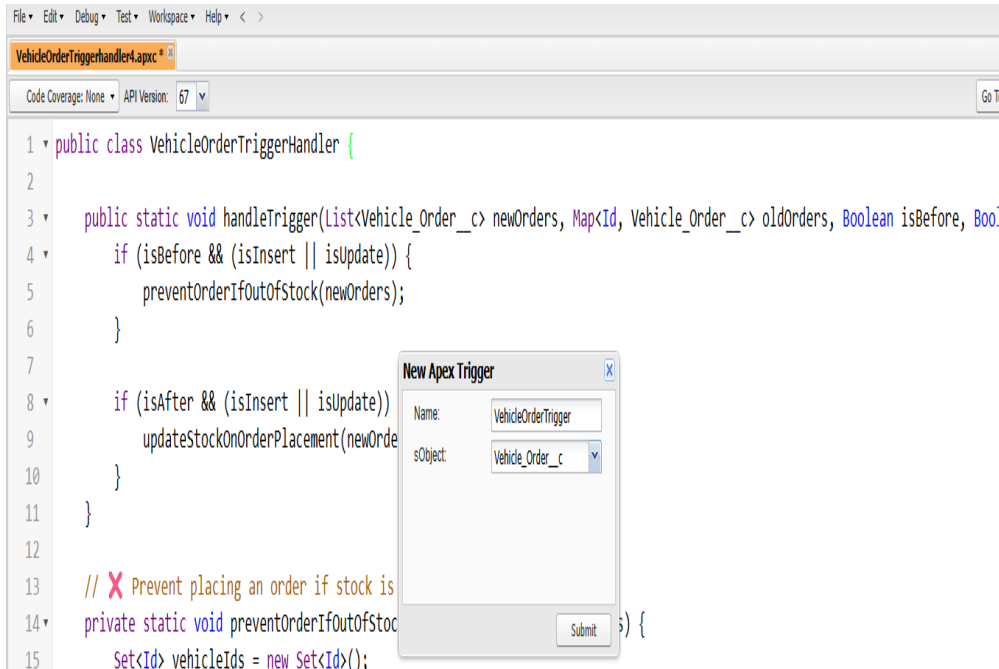
- This vehicle is out of stock. Order cannot be placed.

CancelSave

Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



## Step 7 : Call Apex Class in Trigger Class

Source Code:

trigger VehicleOrderTrigger on Vehicle\_Order\_\_c (before insert, before update, after insert, after update) {

VehicleOrderTriggerHandler.handleTrigger([Trigger.new](#), Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);

}



## Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.

## New Apex class



Please enter a name for your new Apex class

OK

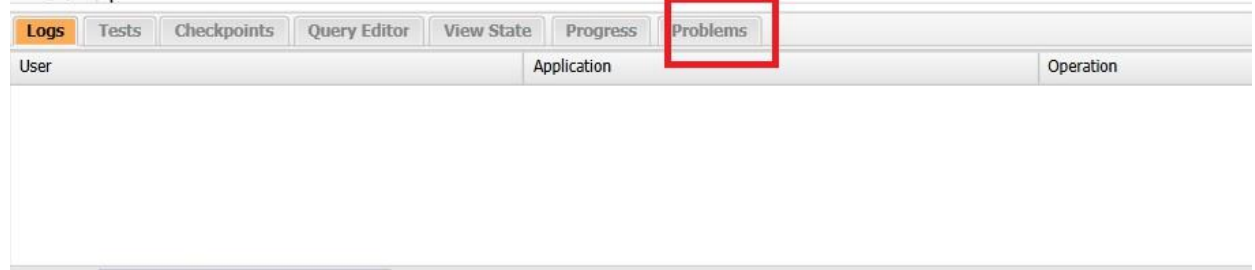
Cancel

Code Coverage: None API Version: 62

```
1  global class VehicleOrderBatch implements Database.Batchable<sObject> {
2
3  global Database.QueryLocator start(Database.BatchableContext bc) {
4  return Database.getQueryLocator([
5      SELECT Id, Status__c, Vehicle__c
6      FROM Vehicle_Order__c
7      WHERE Status__c = 'Pending'
8  ]);
9  }
10
11 global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
12     Set<Id> vehicleIds = new Set<Id>();
13
14     for (Vehicle_Order__c order : orderList) {
15         if (order.Vehicle__c != null) {
16             vehicleIds.add(order.Vehicle__c);
17         }
18     }
19
20     if (!vehicleIds.isEmpty()) {
21         Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
22         for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
23             vehicleStockMap.put(vehicle.Id, vehicle);
24         }
25
26         List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
27         List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
28
29         for (Vehicle_Order__c order : orderList) {
30             if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
30 ▾      if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31          Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32 ▾      if (vehicle.Stock_Quantity__c > 0) {
33          order.Status__c = 'Confirmed';
34          vehicle.Stock_Quantity__c -= 1;
35          ordersToUpdate.add(order);
36          vehiclesToUpdate.add(vehicle);
37      }
38  }
39  }
40
41 ▾      if (!ordersToUpdate.isEmpty()) {
42          update ordersToUpdate;
43      }
44
45 ▾      if (!vehiclesToUpdate.isEmpty()) {
46          update vehiclesToUpdate;
47      }
48  }
49  }
50
51 ▾  global void finish(Database.BatchableContext bc) {
52      System.debug('Vehicle order batch job completed.');
```

53 }  
54 }



The screenshot shows a code editor with a toolbar at the bottom. The toolbar includes tabs for 'Logs', 'Tests', 'Checkpoints', 'Query Editor', 'View State', 'Progress', and 'Problems'. The 'Problems' tab is highlighted with a red rectangle. Below the toolbar, there is a table with columns for 'User', 'Application', and 'Operation'.

## Source Code:

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {  
  
    global Database.QueryLocator start (Database.BatchableContext bc) {  
  
        return Database.getQueryLocator([  
  
            SELECT Id, Status__c, Vehicle__c  
  
            FROM Vehicle_Order__c  
  
            WHERE Status__c
```

'Pending'

]);

}

global void execute(Database.BatchableContext bc, List<Vehicle\_Order\_c> orderList) {

Set<Id> vehicleIds = new Set<Id>();

for (Vehicle\_Order\_c order orderList) {

if (order.Vehicle\_c != null) {

vehicleIds.add(order.Vehicle\_\_c);

}

}

if (!vehicleIds.isEmpty()) {

Map<Id, Vehicle\_\_\_c> vehicleStockMap = new Map<Id, Vehicle\_\_\_c>();

for (Vehicle\_c vehicle: [SELECT Id, Stock Quantity\_c FROM Vehicle\_c WHERE Id IN vehicleIds]) {

vehicleStockMap.put(vehicle. Id, vehicle);

}

List<Vehicle\_Order\_c> ordersToUpdate = new List<Vehicle\_Order\_c>();

List<Vehicle\_\_c> vehiclesToUpdate = new List<Vehicle\_\_c>();

for (Vehicle\_Order\_c order orderList) {

if (vehicleStockMap.containsKey(order. Vehicle\_\_c)) {

Vehicle\_c vehicle = vehicleStockMap.get(order. Vehicle\_\_c);

if (vehicle.Stock\_Quantity\_\_\_c > 0) {

order.Status\_\_\_c = 'Confirmed';

```
vehicle.Stock_Quantity___c -= 1;
```

```
ordersToUpdate.add(order);
```

```
vehiclesToUpdate.add(vehicle);
```

```
}
```

```
}
```

```
}
```

```
if (!ordersToUpdate.isEmpty()) {
```

```
    update ordersToUpdate;
```

```
}
```

```
if (!vehiclesToUpdate.isEmpty()) {
```

```
    update vehiclesToUpdate;
```

```
}
```

```
}
```

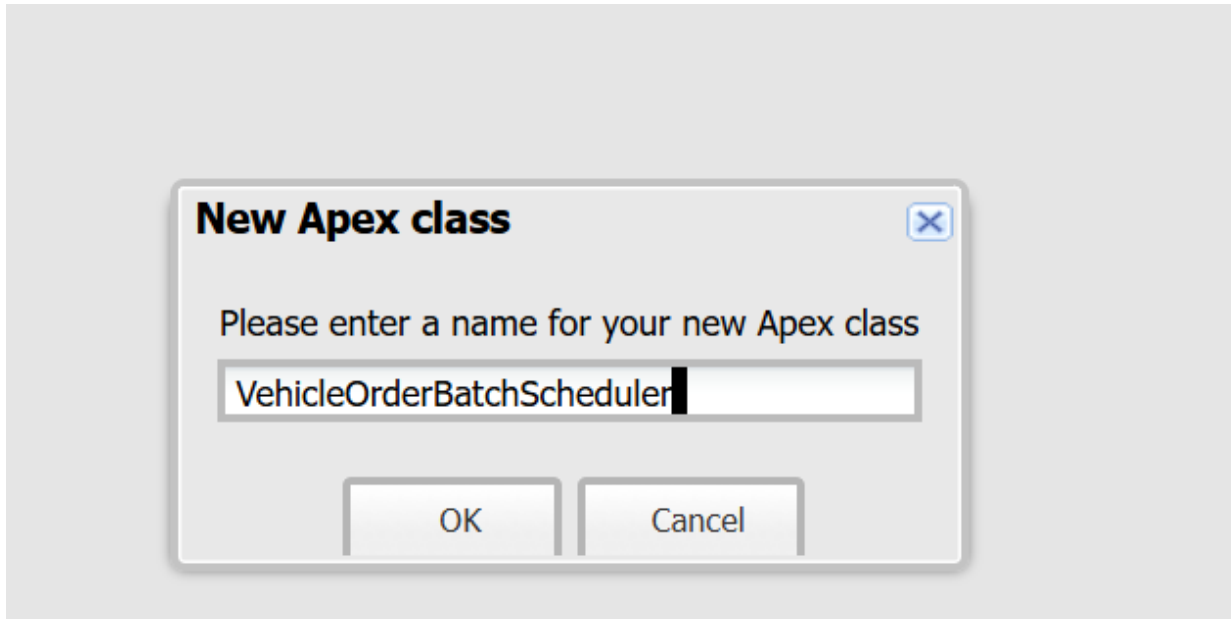
```
global void finish(Database.BatchableContext bc) {
```

```
    System.debug('Vehicle order batch job completed.');
```

```
}
```

```
}
```

**Step 9 :** Create Schedule Class then revoked Batch Class in Schedule Class



```
OrderTriggerHandler.apxc | OrderTrigger.apxt | InventoryBatchJob.apxc | VehicleOrderTriggerHandler.apxc | VehicleOrderTrigger.apxt | VehicleOrderBatchScheduler.apxc
Code Coverage: None | API Version: 62
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
7
```

### Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {

    global void execute(SchedulableContext sc) {

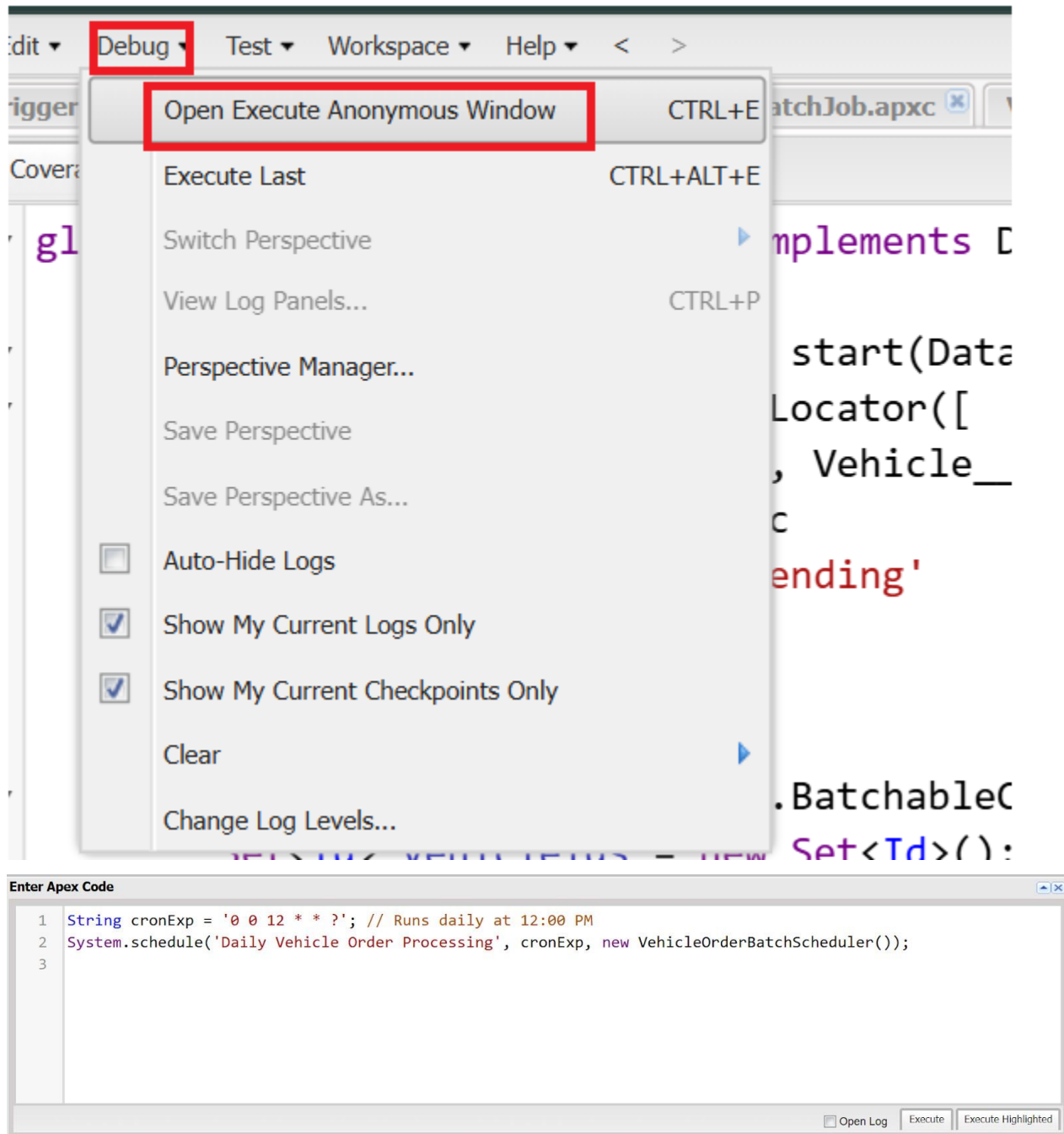
        VehicleOrderBatch batchJob = new VehicleOrderBatch();

        Database.executeBatch(batchJob, 50); // 50 is the batch size

    }

}
```

**Step 10 : Schedule the Batch Job** To run the batch job **every night at midnight** :

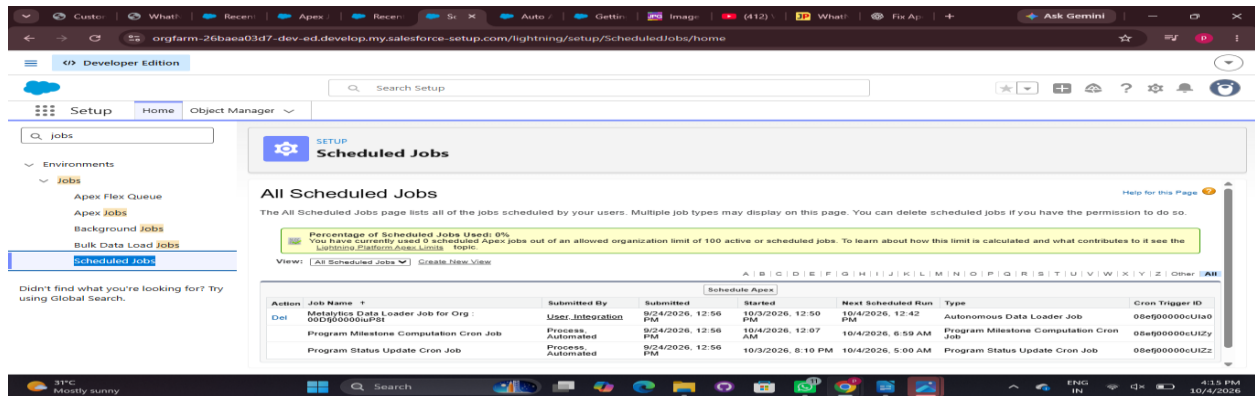


**Source Code :**

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
```

```
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

## Step 11 : You Can Check where this Schedule Job is Running



## 7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

## 8. ADVANTAGES & DISADVANTAGES

### Advantages

- Centralized data
- Automated workflows
- Real-time reporting

### Disadvantages

- Salesforce dependency
- Requires initial configuration effort

## **9. Conclusion:**

The Salesforce CRM implementation at WhatNext Vision Motors successfully modernizes vehicle ordering and dealership management. Automation improves efficiency, accuracy, and customer experience.

## **10. FUTURE SCOPE**

- Mobile ordering app
- Dealer GPS integration
- SMS notifications
- AI-based demand forecasting

## **11. APPENDIX**

- Source Code: Salesforce Configuration
- Dataset: Salesforce Objects
- GitHub / Demo: Salesforce Developer Org

**Thank You**



